SMAI — End-Semester Preparation Guide

Statistical Methods in AI
IIIT Hyderabad — Spring 2026

Compiled from class material, past papers (Q1, Mid-Sem, Q2) and Spring-2025 PYQs

April 29, 2026

How to use this guide. This is a *playbook*, not a textbook. Each topic is structured the same way: (1) **Formulae** you must remember, (2) **Recipe** for solving the typical exam question, (3) one or two **worked examples** in the prof's style, and (4) **Gotchas** – the silly mistakes that cost marks. Past-paper solutions are at the end. *Do* the worked examples by hand before moving on.

Contents

1. Endsem Strategy and the Prof's Style

1.1. Format and weights

- **3 hours, 180 marks, 3 questions.**
- **Q1: 12 short objective questions × 5 = 60 marks.** No working shown; precise answers only. *Time-box:* 60 minutes (5 min each).
- **Q2: Answer 3 of 4 medium questions × 20 = 60 marks.**
- **Q3: Answer 3 of 4 long questions × 20 = 60 marks.**

1.2. What the prof loves to test (recurring across Q1/MS/Q2/PYQ)

- Vector norms (L_0, L_1, L_2, L_∞) on a small vector.
- GD with η given, do 2 iterations, then "best η for one-step convergence".
- Closed form $(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$ on tiny dataset.
- Precision / Recall from a 10-element scored list and a threshold.
- KNN by hand with 4–5 train, 4–5 test, Euclidean or Manhattan.
- PCA / Eigenfaces: covariance size, rank, dimensionality reduction recipe.
- Bayes' theorem on bags-of-balls and Gaussian classifier threshold.
- Kernel proofs: K symmetric, K PSD, find ϕ for $(\mathbf{p}^\top\mathbf{q})^k$.
- SVM: support vectors on a 1D dataset, margin, primal derivation.
- K-Means: 3 iterations on 8–10 given points.
- MLP: forward, $\partial L/\partial w_{ij}$ via chain rule, numerical gradient, SLP-equivalent if ϕ_1 linear.
- Perceptron training algorithm; show 2 iterations.
- Conv layer parameter count; output dimension.
- Cross-entropy numerical value on a one-hot target.
- Self-attention dimensions / numerical pass.
- TRUE/FALSE: BP/global minima, zero init, cross-entropy for regression, residuals fix vanishing grad, L_1 produces sparsity, dropout compresses model, etc.

1.3. Heuristics that buy easy marks

Trick — use these on every paper

- **Always center first, then scale.** For Z-score: $z_i = (x_i - \mu)/\sigma$. Pop std uses $1/N$, sample std uses $1/(N-1)$ — *either* is accepted; just be consistent.
- **Whenever you see "minimize $\mathbf{w}^\top M\mathbf{w}$ s.t. $\|\mathbf{w}\| = 1$":** answer is the eigenvector of the *smallest* eigenvalue of M (Rayleigh quotient). Maximize ⇒ largest. PCA is the maximize case on $\mathbf{X}^\top\mathbf{X}$ (covariance).
- **For $\min \|A\mathbf{x}\|^2$ s.t. $\|\mathbf{x}\| = 1$ with $A : m \times n$ $(m > n)$:** eigenvector of *smallest* eigenvalue of $A^\top A$ (size $n \times n$). Don't pick $AA^\top$ — wrong matrix.
- **Newton step for quadratic $f(w) = aw^2 + bw + c$:** $\eta^* = 1/f''(w) = 1/(2a)$ converges in one step from any starting point. *Do this whenever the prof asks "one-step" η.*
- **Param count, MLP, no bias, $d_{\text{in}} \to h \to d_{\text{out}}$:** $d_{\text{in}} \cdot h + h \cdot d_{\text{out}}$. With bias: add $h + d_{\text{out}}$.
- **Conv-1D params:** $k \cdot C_{\text{in}} \cdot C_{\text{out}}$ ($+C_{\text{out}}$ for bias).
- **Output length 1D conv:** $\lfloor (L + 2P - k)/s \rfloor + 1$.
- **Two distributions $\mathcal{N}(\mu_1, \sigma_1^2), \mathcal{N}(\mu_2, \sigma_2^2)$:** Bayes threshold is shifted *toward the tighter* (smaller-σ) one — its tail decays faster.

- **Linear MLP collapses to SLP.** If ϕ_1 is identity, $o = \mathbf{u}^\top \mathbf{W}\mathbf{x} = (\mathbf{W}^\top \mathbf{u})^\top \mathbf{x}$. Compute $\mathbf{v} = \mathbf{W}^\top \mathbf{u}$ to get the equivalent SLP weights.
- **Cross-entropy with one-hot target:** $L = -\log(\hat{y}_{\text{class}})$ — only the true class entry contributes. *Numerical:* read off the predicted prob for the correct class.

1.4. Time budget for a typical 20-mark Q3-style problem

- 2 min reading and identifying the recipe.
- 2 min sketching what each sub-part wants.
- 12–14 min computing.
- 2 min sanity check (dimensions match? sign correct? gradient direction?).

If you're stuck after 10 min, *move on*; come back. A blank Q3 costs 20; a half Q3 might still earn 8–12.

2. Master Formula Reference

A one-stop sheet. Write the formula, then expand on it later. *Memorise these in this order.*

Norms (vector $\mathbf{x} \in \mathbb{R}^d$)

$$L_0 = \#\{i : x_i \neq 0\}, \quad L_1 = \sum |x_i|, \quad L_2 = \sqrt{\sum x_i^2}, \quad L_\infty = \max_i |x_i|.$$

Linear regression

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}, \qquad J(\mathbf{w}) = \tfrac{1}{N} \sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)^2.$$

GD: $\mathbf{w}^{k+1} = \mathbf{w}^k + \frac{2\eta}{N} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}^k)$.

Regularisation

Ridge: $J = \frac{1}{N} \sum (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_2^2, \qquad \hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \lambda I)^{-1} \mathbf{X}^\top \mathbf{y}.$

LASSO: $J = \frac{1}{N} \sum (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_1$. (no closed form; sparsifies)

Covariance & PCA

$$\boldsymbol{\mu} = \tfrac{1}{N} \sum_i \mathbf{x}_i, \qquad \boldsymbol{\Sigma} = \tfrac{1}{N} \sum_i (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^\top \text{ (or } \tfrac{1}{N-1}).$$

PCs = eigenvectors of $\boldsymbol{\Sigma}$ corresponding to top k eigenvalues. **PCA via SVD:** $\mathbf{X}_c = U \Sigma V^\top \Rightarrow$ PCs are columns of V; explained variance $= \sigma_i^2/(N-1)$. **PCA trick** (when $d \gg N$): work with $\mathbf{X}_c \mathbf{X}_c^\top$ $(N \times N)$ instead of $\mathbf{X}_c^\top \mathbf{X}_c$; recover full eigenvectors as $\mathbf{u} = \mathbf{X}_c^\top \mathbf{v}$ (then normalise).

Bayes & Gaussians

$$P(\omega_i \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid \omega_i) P(\omega_i)}{p(\mathbf{x})}.$$

Decision: pick ω_i with largest posterior. For 1D Gaussians with equal σ, threshold is the midpoint $(\mu_1 + \mu_2)/2$. With unequal σ, threshold shifts *toward the tighter* class.

Logistic / softmax

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}, \quad L_{\text{CE}} = -\sum_i y_i \log \hat{y}_i.$$

For one-hot y: $L = -\log \hat{y}_{\text{true}}$.

SVM

Primal (hard): $\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2$ s.t. $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$.
Soft margin: $\min \frac{1}{2}\|\mathbf{w}\|^2 + C \sum \xi_i$ s.t. $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \xi_i \geq 0$.
Margin $= 2/\|\mathbf{w}\|$. Support vectors = points with $\alpha_i > 0$ (lie on margin or violate it).

Kernel trick

$\kappa(\mathbf{p}, \mathbf{q}) = \phi(\mathbf{p})^\top \phi(\mathbf{q})$. $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$. K is symmetric and PSD.
Polynomial: $(\mathbf{p}^\top \mathbf{q} + c)^d$. Gaussian / RBF: $\exp(-\gamma \|\mathbf{p} - \mathbf{q}\|^2)$.

K-Means

$$J = \sum_{k=1}^{K} \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2, \quad \boldsymbol{\mu}_k = \tfrac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} \mathbf{x}, \quad C(\mathbf{x}_i) = \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2.$$

Not convex — multiple restarts; converges to local minimum.

Perceptron

$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$. **Update only on misclassified samples**: $\mathbf{w} \leftarrow \mathbf{w} + \eta y \mathbf{x}$, $b \leftarrow b + \eta y$.

Backprop on a generic chain

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial q_j} \cdot \frac{\partial q_j}{\partial p_j} \cdot \frac{\partial p_j}{\partial w_{ij}}.$$

For ReLU: $\phi'(p) = \mathbf{1}[p > 0]$. For linear: $\phi'(p) = 1$. For sigmoid: $\sigma(p)(1 - \sigma(p))$.

Optimisers

Plain GD: $\mathbf{w}^{k+1} = \mathbf{w}^k - \eta \nabla L$.
Momentum: $\mathbf{v}^{k+1} = \beta \mathbf{v}^k + \nabla L$, $\mathbf{w}^{k+1} = \mathbf{w}^k - \eta \mathbf{v}^{k+1}$ ($\beta \in [0, 1)$).
Newton: $\mathbf{w}^{k+1} = \mathbf{w}^k - [H^k]^{-1} \nabla L$ (H = Hessian).
Adam: $m = \beta_1 m + (1 - \beta_1) g$, $v = \beta_2 v + (1 - \beta_2) g^2$, $\mathbf{w} \leftarrow \mathbf{w} - \eta \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$.

Convolution arithmetic (1D / 2D)

Output spatial size: $\lfloor (L + 2P - k)/s \rfloor + 1$.
Params per conv layer: $k_h \cdot k_w \cdot C_{\text{in}} \cdot C_{\text{out}}$ ($+C_{\text{out}}$ for bias).
For 1D: $k \cdot C_{\text{in}} \cdot C_{\text{out}}$ ($+C_{\text{out}}$).

Self-Attention (one head)

$Q = XW_Q$, $K = XW_K$, $V = XW_V$ (all $n \times d_k$).
Attention: $\text{softmax}(QK^\top / \sqrt{d_k})V$.
Multi-head: concat h heads, then W_O. Each head's matrices are $d \times d_h$.

3. Topic 1 — Linear Algebra Essentials (the hidden currency)

3.1. Rank, eigenvalues, eigenvectors

- Product of eigenvalues $= \det(A)$. Sum of eigenvalues $= \mathrm{tr}(A)$.
- For $m \times n$ matrix A, $\mathrm{rank}(A) \leq \min(m, n)$.
- If A is symmetric, eigenvectors are orthogonal — *but not in general.*
- Covariance $\mathbf{\Sigma}$ is symmetric *and* PSD (eigenvalues ≥ 0).
- $\mathbf{\Sigma}$ is diagonal $\Leftrightarrow$ features are uncorrelated; for a Gaussian, this means independence.

3.2. Singular Value Decomposition (SVD)

For any $A \in \mathbb{R}^{m \times n}$:

$$A = U\,D\,V^\top, \qquad U : m \times m,\ D : m \times n,\ V : n \times n.$$

U orthonormal columns (*left singular vectors* = eigenvectors of $AA^\top$);
V orthonormal columns (*right singular vectors* = eigenvectors of $A^\top A$);
D diagonal with singular values $\sigma_i = \sqrt{\lambda_i}$ (eigenvalues of $A^\top A$ or $AA^\top$).

Recipe — compute SVD of a 2×2 matrix

1. Compute $A^\top A$ (size $n \times n$).
2. Find its eigenvalues λ_i and unit eigenvectors $\mathbf{v}_i$.
3. Singular values: $\sigma_i = \sqrt{\lambda_i}$. Order from largest.
4. V has $\mathbf{v}_i$ as columns. D is diagonal with σ_i.
5. U: column i is $\mathbf{u}_i = (1/\sigma_i)A\mathbf{v}_i$.
6. Sanity-check $UDV^\top = A$.

Example 3.2 — SVD of $A = \begin{bmatrix} 4 & 4 \\ -3 & 3 \end{bmatrix}$

Step 1. $A^\top A = \begin{bmatrix} 16+9 & 16-9 \\ 16-9 & 16+9 \end{bmatrix} = \begin{bmatrix} 25 & 7 \\ 7 & 25 \end{bmatrix}$.

Step 2. Eigenvalues: characteristic poly $(25-\lambda)^2 - 49 = 0 \Rightarrow \lambda = 25 \pm 7 = 32,\ 18$.

For $\lambda = 32$: $(A^\top A - 32I)\mathbf{v} = 0 \Rightarrow \begin{bmatrix} -7 & 7 \\ 7 & -7 \end{bmatrix}\mathbf{v} = 0 \Rightarrow \mathbf{v}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

For $\lambda = 18$: $\mathbf{v}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

Step 3. $\sigma_1 = \sqrt{32} = 4\sqrt{2},\ \sigma_2 = \sqrt{18} = 3\sqrt{2}$.

Step 4. $V = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix},\ D = \begin{bmatrix} 4\sqrt{2} & 0 \\ 0 & 3\sqrt{2} \end{bmatrix}$.

Step 5. $\mathbf{u}_1 = \frac{1}{4\sqrt{2}}A\mathbf{v}_1 = \frac{1}{4\sqrt{2}} \cdot \frac{1}{\sqrt{2}}\begin{bmatrix} 8 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. $\quad \mathbf{u}_2 = \frac{1}{3\sqrt{2}} \cdot \frac{1}{\sqrt{2}}\begin{bmatrix} 0 \\ -6 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$.

Result. $U = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix},\ D = \begin{bmatrix} 4\sqrt{2} & 0 \\ 0 & 3\sqrt{2} \end{bmatrix},\ V = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$.

Recurring Q: SVD of $A = aI$

$A = aI \Rightarrow U = I,\ D = |a|I,\ V = \mathrm{sign}(a) \cdot I$. (Any orthogonal U, V that satisfy $UV^\top = \mathrm{sign}(a)I$ works.)

3.3. Solving an over-determined homogeneous system

Question: $A\mathbf{x} = \mathbf{0}$ with A ($m \times n$, $m > n$); find optimal $\mathbf{x}$ s.t. $\|\mathbf{x}\| = 1$.
Equivalent to $\min \|A\mathbf{x}\|^2 = \min \mathbf{x}^\top A^\top A\mathbf{x}$ on the unit sphere.
Answer: **eigenvector of the smallest eigenvalue of $A^\top A$.** (Not $AA^\top$ — the wrong size.)

Example 3.3 — fit a line through 3 points (homogeneous form)

Find the line $ax + by + c = 0$ closest to the points $(1,1), (2,2), (3,3.1)$.
Stack as $A = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 1 \\ 3 & 3.1 & 1 \end{bmatrix}$ (3×3, but treat the parameters $\mathbf{x} = [a, b, c]^\top$ as the unknown with $\|\mathbf{x}\| = 1$).
$A^\top A$ is 3×3. Compute its smallest eigenvalue's eigenvector — that's $\mathbf{x}$. By inspection the points are nearly on $y = x \Rightarrow x - y = 0 \Rightarrow \mathbf{x} \propto [1, -1, 0]^\top$. After unit-normalising: $\mathbf{x} = \frac{1}{\sqrt{2}}[1, -1, 0]^\top$.
Why $A^\top A$ and not $AA^\top$? $A^\top A$ is 3×3 — same as the dimension of $\mathbf{x}$. $AA^\top$ would be 3×3 here too because $m = n$, but in general (e.g. $m = 100, n = 3$) $AA^\top$ is 100×100 — wrong size.

3.4. Low-rank approximation

The best rank-k approximation (in Frobenius norm) of A is $A_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$, obtained by zeroing all but the top-k singular values.

Example 3.4 — rank-1 approximation of the matrix from 3.2

$A = \begin{bmatrix} 4 & 4 \\ -3 & 3 \end{bmatrix}$ has $\sigma_1 = 4\sqrt{2}, \sigma_2 = 3\sqrt{2}$. The rank-1 approximation drops σ_2:

$$A_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top = 4\sqrt{2} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} \cdot \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} = 4 \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 4 \\ 0 & 0 \end{bmatrix}.$$

Reconstruction error: $\|A - A_1\|_F = \sqrt{0 + 0 + 9 + 9} = \sqrt{18} = \sigma_2$ (always — Eckart-Young).

3.5. Worked example — practice

Example 3.1

A vector $\mathbf{x} = [-4, 2, 0, 4]^\top$. Find all four norms.

$$\begin{aligned} L_0 &= 3 \text{ (three non-zero entries)} \\ L_1 &= |-4| + |2| + |0| + |4| = 10 \\ L_2 &= \sqrt{16 + 4 + 0 + 16} = \sqrt{36} = 6 \\ L_\infty &= \max\{4, 2, 0, 4\} = 4 \end{aligned}$$

Gotcha

L_0 is *not* a norm in the strict sense (it isn't homogeneous), but the prof always treats it as "count of non-zero elements". Use that.

4. Topic 2 — Linear Regression & Regularisation

4.1. Closed-form recipe (do this every time)

Recipe — Closed-form regression

Given $\{(x_i, y_i)\}$ and target $y_i = w_0 + w_1 x_i$ (or general $\mathbf{w}^\top \mathbf{x}$):

1. Build $\mathbf{X}$ with a column of 1s for the bias: $\mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \end{bmatrix}$.
2. Build $\mathbf{y}$ as a column vector.
3. $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.
4. Read w_0 (intercept) and w_1 (slope).

4.2. Worked example (from Quiz 1, Set 1)

Example 4.1 — $\{(1,2),(2,3),(3,4)\}$

$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$, $\mathbf{y} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$. $\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}$, $\det = 42 - 36 = 6$, $(\mathbf{X}^\top \mathbf{X})^{-1} = \frac{1}{6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix}$.

$\mathbf{X}^\top \mathbf{y} = \begin{bmatrix} 9 \\ 20 \end{bmatrix} \Rightarrow \mathbf{w}^* = \frac{1}{6} \begin{bmatrix} 14 \cdot 9 - 6 \cdot 20 \\ -6 \cdot 9 + 3 \cdot 20 \end{bmatrix} = \frac{1}{6} \begin{bmatrix} 6 \\ 6 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Answer: $y = 1 + x$.

Faster on visible patterns

If the data lies exactly on a line, just *look*. For $\{(1,2),(2,3),(3,4),(4,5)\}$, $y = x + 1 \Rightarrow w_1 = 1, w_0 = 1$. The prof rewards this — see PYQ Q2.4(i): "without numerically computing, guess $\mathbf{w}$ by observing the data".

4.3. Gradient descent for MSE

$$\nabla_{\mathbf{w}} L = -\frac{2}{N} \sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i) \mathbf{x}_i$$

$$\mathbf{w}^{k+1} = \mathbf{w}^k + \frac{2\eta}{N} \sum_i (y_i - \mathbf{w}^{k\top} \mathbf{x}_i) \mathbf{x}_i.$$

Matrix form: $\mathbf{w}^{k+1} = \mathbf{w}^k + \frac{2\eta}{N} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}^k)$.

Example 4.2 — two GD iterations on the regression dataset

Same $\{(1,2),(2,3),(3,4)\}$. Start from $\mathbf{w}^0 = [0,0]^\top$, $\eta = 0.05$.

$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$, $\mathbf{y} = [2,3,4]^\top$. Predictions $\mathbf{X}\mathbf{w}^0 = [0,0,0]^\top$.

Residual $\mathbf{y} - \mathbf{X}\mathbf{w}^0 = [2,3,4]^\top$. $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}^0) = [2+3+4,\ 2+6+12]^\top = [9,20]^\top$.

Iteration 1: $\mathbf{w}^1 = [0,0] + \frac{2 \cdot 0.05}{3} [9,20] = [0,0] + [0.3, 0.667] = [0.3, 0.667]^\top$.

Predictions $\mathbf{X}\mathbf{w}^1 = [0.967, 1.633, 2.3]^\top$. Residual $= [1.033, 1.367, 1.7]^\top$.

$\mathbf{X}^\top \text{res} = [4.1, 9.067]^\top$.

Iteration 2: $\mathbf{w}^2 = [0.3, 0.667] + \frac{0.1}{3} [4.1, 9.067] = [0.437, 0.969]^\top$.

Heading toward the closed-form answer $[1,1]^\top$.

4.4. Ridge vs LASSO at a glance

	Ridge (L_2)	LASSO (L_1)
Penalty	$\lambda\|\mathbf{w}\|_2^2$	$\lambda\|\mathbf{w}\|_1$
Closed form?	Yes: $(\mathbf{X}^\top\mathbf{X}+\lambda I)^{-1}\mathbf{X}^\top\mathbf{y}$	No (sub-gradient / coord. descent)
Effect	Shrinks weights	Drives weights to *exact* zero $\Rightarrow$ feature selection
Geometry	Spherical constraint	Diamond constraint (corners on axes)

4.5. General weighted regression (PYQ classic)

$J(\boldsymbol{\theta}) = (\mathbf{y}-\mathbf{X}\boldsymbol{\theta})^\top A(\mathbf{y}-\mathbf{X}\boldsymbol{\theta})$ with A symmetric.
Setting $\partial J/\partial\boldsymbol{\theta} = 0$ gives $\boldsymbol{\theta} = (\mathbf{X}^\top A\mathbf{X})^{-1}\mathbf{X}^\top A\mathbf{y}$.

Example 4.3 — Ridge regression with $\lambda = 1$

Same data $\{(1,2),(2,3),(3,4)\}$, but with ridge.

$\mathbf{X}^\top\mathbf{X}+\lambda I = \begin{bmatrix}3 & 6\\ 6 & 14\end{bmatrix} + \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix} = \begin{bmatrix}4 & 6\\ 6 & 15\end{bmatrix}$, $\det = 60-36 = 24$.

$(\mathbf{X}^\top\mathbf{X}+\lambda I)^{-1} = \frac{1}{24}\begin{bmatrix}15 & -6\\ -6 & 4\end{bmatrix}$.

$\mathbf{X}^\top\mathbf{y} = [9, 20]^\top$.

$\mathbf{w} = \frac{1}{24}\begin{bmatrix}15\cdot 9 - 6\cdot 20\\ -6\cdot 9 + 4\cdot 20\end{bmatrix} = \frac{1}{24}\begin{bmatrix}15\\ 26\end{bmatrix} = \begin{bmatrix}0.625\\ 1.083\end{bmatrix}$.

Compare with the un-regularised $[1,1]$: ridge has shrunk the L2 norm of $\mathbf{w}$ from $\sqrt{2}\approx 1.41$ down to $\sqrt{0.625^2+1.083^2}\approx 1.25$, as expected. The line is no longer the perfect interpolator — that's the bias-variance trade-off.

5. Topic 3 — PCA, SVD, Eigenfaces

5.1. Step-by-step PCA

Recipe — PCA on data $\mathbf{X}\in\mathbb{R}^{N\times d}$

1. Compute mean $\boldsymbol{\mu} = \frac{1}{N}\sum_i \mathbf{x}_i$.
2. Centre: $\mathbf{X}_c = \mathbf{X} - \mathbf{1}\boldsymbol{\mu}^\top$.
3. Compute covariance $\boldsymbol{\Sigma} = \frac{1}{N}\mathbf{X}_c^\top\mathbf{X}_c$ (size $d\times d$).
4. Eigendecompose $\boldsymbol{\Sigma}$; sort eigenvalues decreasing.
5. Pick top-k eigenvectors $\mathbf{w}_1,\dots,\mathbf{w}_k$ (form columns of $\mathbf{W}$).
6. Project: $\mathbf{z}_i = \mathbf{W}^\top(\mathbf{x}_i-\boldsymbol{\mu})\in\mathbb{R}^k$.

5.2. Eigenfaces / large d trick

When $d \gg N$ (e.g. $d = 10000$ pixels, $N = 200$ images), $\boldsymbol{\Sigma}$ is huge but *rank-deficient* (rank $\le N-1$). Use the trick:

$$\mathbf{X}_c\mathbf{X}_c^\top \text{ (small, } N\times N) \xrightarrow{\text{eigendecomp}} \mathbf{v} \longrightarrow \mathbf{u} = \mathbf{X}_c^\top\mathbf{v} \text{ (normalise to unit length).}$$

$\mathbf{u}$ is the eigenvector of the full $\boldsymbol{\Sigma} = \frac{1}{N}\mathbf{X}_c^\top\mathbf{X}_c$ for the same eigenvalue.

5.3. Choose k by explained variance

$$k = \min\left\{k : \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^N \lambda_i} \ge 0.95\right\}.$$

Example 5.1 — full PCA on a tiny 2D dataset

Data: $\{(2,0),(0,2),(-2,0),(0,-2)\}$. Find PC1.
Mean: $\boldsymbol{\mu} = (0,0)$ (already centred).
Covariance: $\boldsymbol{\Sigma} = \frac{1}{4}\sum \mathbf{x}_i\mathbf{x}_i^\top = \frac{1}{4}\left(\begin{bmatrix}4&0\\0&0\end{bmatrix}+\begin{bmatrix}0&0\\0&4\end{bmatrix}+\begin{bmatrix}4&0\\0&0\end{bmatrix}+\begin{bmatrix}0&0\\0&4\end{bmatrix}\right) = \begin{bmatrix}2&0\\0&2\end{bmatrix}$.
Eigenvalues: $\lambda_1 = \lambda_2 = 2$. Both directions have equal variance — PCA is degenerate (rotational symmetry). Any unit vector is a PC.
Now add asymmetry: $\{(2,0),(-2,0),(1,1),(-1,-1)\}$.
$\boldsymbol{\Sigma} = \frac{1}{4}\left(\begin{bmatrix}4&0\\0&0\end{bmatrix}+\begin{bmatrix}4&0\\0&0\end{bmatrix}+\begin{bmatrix}1&1\\1&1\end{bmatrix}+\begin{bmatrix}1&1\\1&1\end{bmatrix}\right) = \begin{bmatrix}2.5&0.5\\0.5&0.5\end{bmatrix}$.
Eigenvalues from $\det(\boldsymbol{\Sigma}-\lambda I) = (2.5-\lambda)(0.5-\lambda)-0.25 = \lambda^2-3\lambda+1 = 0 \Rightarrow \lambda = (3\pm\sqrt{5})/2 \approx 2.618,\ 0.382$.
Explained variance: PC1 explains $2.618/3 \approx 87\%$ — keeping $k=1$ is enough.
PC1 direction: $(\boldsymbol{\Sigma}-\lambda_1 I)\mathbf{v} = 0 \Rightarrow \mathbf{v}_1 \propto [1, 2-\sqrt{5}]^\top \approx [1,-0.236]$, normalised $\mathbf{v}_1 \approx [0.973,-0.230]^\top$.

Example 5.2 — PCA via gradient ascent

For $\boldsymbol{\Sigma} = \begin{bmatrix}2&1\\1&2\end{bmatrix}$, start from $\mathbf{v}^0 = [1,0]^\top$, $\eta = 0.1$.
$\mathbf{v}^1 = (I+\eta\boldsymbol{\Sigma})\mathbf{v}^0 = \begin{bmatrix}1.2&0.1\\0.1&1.2\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} = [1.2, 0.1]^\top$.
Normalise: $\|\mathbf{v}^1\| = \sqrt{1.44+0.01} \approx 1.204 \Rightarrow \mathbf{v}^1 \approx [0.997, 0.083]^\top$.
After several iterations $\mathbf{v} \to [1/\sqrt{2}, 1/\sqrt{2}]$, the eigenvector of $\lambda_1 = 3$.

5.4. Recurring exam facts (memorise)

- Covariance matrix size = *feature dimension* × feature dimension ($d \times d$, never $N \times N$). For 1000 images of 100×100: $\boldsymbol{\Sigma}$ is 10000×10000.
- Practical rank $\ll \min(N, d)$ — most variance is in a few components.
- If all images were identical, $\boldsymbol{\Sigma}$ would be **zero everywhere**; the mean equals that single image.
- If images are N images of K individuals ($K \ll N$), expect rank close to K (or $K-1$ after centering).
- A *fixed* geometric transform of all data does not change the rank. *Random per-image* transforms do increase the rank.
- Mean centring is essential — otherwise the first PC just points to the mean.

5.5. PCA via gradient ascent (recurring multiple-choice)

The first PC maximises $\mathbf{v}^\top\boldsymbol{\Sigma}\mathbf{v}$ s.t. $\|\mathbf{v}\| = 1$. Iterative update:

$$\mathbf{v}^{k+1} = (I + \eta\boldsymbol{\Sigma})\,\mathbf{v}^k \quad \text{(normalise after each step)}.$$

5.6. Eigenfaces classification — full recipe (PYQ-style)

Recipe — Face recognition with PCA + KNN

Train: 200 images, 100×100 each ($d = 10000$).
1. Flatten each image to a row of $\mathbf{X} \in \mathbb{R}^{200\times 10000}$.
2. $\boldsymbol{\mu} = \frac{1}{200}\sum_i \mathbf{x}_i$; $\mathbf{X}_c = \mathbf{X} - \mathbf{1}\boldsymbol{\mu}^\top$.

3. Compute $C_{\text{small}} = \frac{1}{200}\mathbf{X}_c\mathbf{X}_c^\top$ (200×200).
4. Eigendecompose; sort eigenvalues; pick k from cum. var. ≥ 0.95.
5. Recover full eigvecs: $\mathbf{U} = \mathbf{X}_c^\top\mathbf{V}$, normalise columns.
6. $\mathbf{Z}_{\text{train}} = \mathbf{X}_c\,\mathbf{U}_{:,1:k}$.
7. For test image Q: flatten, $\mathbf{z}_q = \mathbf{U}_{:,1:k}^\top(Q - \boldsymbol{\mu})$.
8. KNN: distances $\|\mathbf{Z}_{\text{train}} - \mathbf{z}_q\|$, take 5 nearest, majority vote.

6. Topic 4 — Bayesian Decision Theory

6.1. Core formula

$P(\omega_i \mid \mathbf{x}) = \dfrac{p(\mathbf{x} \mid \omega_i)P(\omega_i)}{p(\mathbf{x})}$. Decide ω_i with the largest posterior.

6.2. Two Gaussians, equal priors, equal σ

Threshold is $(\mu_1 + \mu_2)/2$ (the midpoint).

Example 6.0 — Gaussian Bayes classifier numerical

$\mathcal{N}(20, 25)$ vs $\mathcal{N}(30, 5)$, equal priors. Test point $x = 25.1$, predict.
Posteriors $\propto$ likelihoods (priors equal):
$p(25.1 \mid \omega_1) = \frac{1}{\sqrt{2\pi\cdot 25}}\exp(-(25.1-20)^2/(2\cdot 25)) = \frac{1}{\sqrt{50\pi}}e^{-0.520} \approx 0.080 \cdot 0.595 = 0.0475$.
$p(25.1 \mid \omega_2) = \frac{1}{\sqrt{2\pi\cdot 5}}\exp(-(25.1-30)^2/(2\cdot 5)) = \frac{1}{\sqrt{10\pi}}e^{-2.401} \approx 0.179 \cdot 0.0906 = 0.0162$.
$\Rightarrow$ Predict ω_1 (class with $\mu = 20$).
Why? Even though 25.1 is between the means, $\sigma_1 = 5$ is wider — its tail still has reasonable density at 25.1; $\sigma_2 = \sqrt{5}$ is tight and decays much faster. Reinforces the rule: *threshold pulled toward tighter class* (here ω_2 is tighter, so threshold sits between 25.1 and 30 — verifying the prediction).

6.3. Two Gaussians, equal priors, *unequal* σ

The Bayes threshold is shifted *toward the tighter* (smaller σ) class:

- The flatter, wider Gaussian's tail decays slowly and "invades" the tighter class's region.
- So the equal-likelihood crossing happens *closer to the tighter mean.*

Quick sanity check: if you're unsure, draw two bell curves of different widths and see where they cross.

6.4. Worked example (PYQ-style)

Example 6.1

$\mathcal{N}(20, \sigma^2)$ vs $\mathcal{N}(30, 2\sigma^2)$, equal priors. Threshold?
The class at $\mu = 30$ is wider. The wider curve's tail invades the region near 20, so the threshold is *closer to 20.* Mathematically (ln-likelihood equal): $x^* = 10 + \sqrt{200 + 2\sigma^2 \ln 2}$, e.g. for $\sigma = 1$: $x^* \approx 24.19$.

6.5. Bayes' theorem on bags-of-balls (canonical PYQ)

Example 6.2 — Bag I has w_1 white, b_1 black; Bag II has w_2 white, b_2 black

A bag is chosen uniformly, a white ball is drawn. $P(\text{Bag I} \mid W)$?

$$P(\text{Bag I}) = P(\text{Bag II}) = \tfrac{1}{2}.$$
$$P(W \mid \text{I}) = \tfrac{w_1}{w_1+b_1}, \quad P(W \mid \text{II}) = \tfrac{w_2}{w_2+b_2}.$$
$$P(\text{I} \mid W) = \frac{P(W \mid \text{I})P(\text{I})}{P(W \mid \text{I})P(\text{I}) + P(W \mid \text{II})P(\text{II})} = \frac{\frac{w_1}{w_1+b_1}}{\frac{w_1}{w_1+b_1} + \frac{w_2}{w_2+b_2}}.$$

6.6. Naive Bayes (categorical features)

$\hat{y} = \arg\max_y P(y) \prod_i P(x_i \mid y)$. Assumes feature independence given the class. Use log to avoid underflow. Smoothing: add 1 to counts (Laplace).

Example 6.3 — text classification with Naive Bayes

Training: 3 **spam** docs containing {*buy, buy, free*; *free, free, money*; *buy, money*} and 2 **ham** docs {*hello, friend*; *hello, hello*}. Vocab V = {buy,free,money,hello,friend}.
Priors: $P(\text{spam}) = 3/5 = 0.6$, $P(\text{ham}) = 2/5 = 0.4$.
Word counts (spam): buy=3, free=3, money=2, hello=0, friend=0; total=8.
Word counts (ham): buy=0, free=0, money=0, hello=3, friend=1; total=4.
With Laplace ($\alpha = 1$, $|V| = 5$): $P(w \mid \text{spam}) = (\text{count} + 1)/(8 + 5) = (\text{count} + 1)/13$. Similarly $P(w \mid \text{ham}) = (\text{count} + 1)/9$.
Test: "*free money*".
$P(\text{spam} \mid \text{doc}) \propto 0.6 \cdot \frac{4}{13} \cdot \frac{3}{13} = 0.6 \cdot 0.071 = 0.0426$.
$P(\text{ham} \mid \text{doc}) \propto 0.4 \cdot \frac{1}{9} \cdot \frac{1}{9} = 0.0049$.
$\Rightarrow$ Predict **spam**.

7. Topic 5 — Logistic, Softmax, Cross-Entropy

7.1. Binary logistic regression

$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^\top \mathbf{x}}}$; $L_{\text{BCE}} = -\frac{1}{N}\sum_i [y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)]$.

Gradient (clean): $\nabla_{\mathbf{w}} L = \frac{1}{N}\sum_i (\hat{y}_i - y_i)\mathbf{x}_i$. Update: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$.

Example 7.0 — one logistic-regression update

Single sample $\mathbf{x} = [1, 2]^\top$, $y = 1$, init $\mathbf{w} = [0, 0]^\top$, $\eta = 1$.
$z = 0$, $\hat{y} = \sigma(0) = 0.5$.
$\nabla L = (0.5 - 1)\mathbf{x} = -0.5\,[1, 2]^\top = [-0.5, -1]^\top$.
$\mathbf{w}^1 = \mathbf{w}^0 - \eta \nabla L = [0, 0] - [-0.5, -1] = [0.5, 1]^\top$.
Check: new $z = 0.5 + 2 = 2.5$, $\hat{y} = \sigma(2.5) \approx 0.924$ — closer to the target 1.

7.2. Softmax + multinomial CE

$\hat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$; $L_{\text{CE}} = -\sum_c y_c \log \hat{y}_c$. For one-hot y: $L = -\log \hat{y}_{\text{true}}$.

Example 7.2 — softmax forward + CE gradient

Logits $\mathbf{z} = [2, 1, -1]$, true class = 1 (one-hot $\mathbf{y} = [1, 0, 0]$).
$e^z = [7.389, 2.718, 0.368]$, sum = 10.475. $\hat{\mathbf{y}} = [0.706, 0.260, 0.035]$.
$L = -\log(0.706) \approx 0.348$.
Gradient of CE w.r.t. logits: the famous clean form $\partial L/\partial \mathbf{z} = \hat{\mathbf{y}} - \mathbf{y} = [-0.294, 0.260, 0.035]$.
That's all — backprop further by chain rule.

7.3. Worked example (PYQ EndSem 2025 Q1.1)

Example 7.1

NN outputs $[0.2, 0.1, 0.7, 0.0]$, target one-hot $[0, 0, 1, 0]$.
$L = -\log(0.7) \approx 0.357$. The other three terms vanish because their $y_i = 0$.

7.4. Why CE for classification, not regression

- CE pairs naturally with softmax: gradient is $\hat{y} - y$ (clean, non-saturating).
- For regression, MSE is correct because output is unbounded; CE assumes a probability distribution.

8. Topic 6 — SVM and Kernels

8.1. Geometric derivation (tested as a 20-mark Q)

Recipe — derive the SVM primal

1. Hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ separates classes if $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 0$.
2. Distance from $\mathbf{x}_i$ to plane: $|\mathbf{w}^\top \mathbf{x}_i + b|/\|\mathbf{w}\|$.
3. Scale $(\mathbf{w}, b)$ so that the closest points satisfy $\mathbf{w}^\top \mathbf{x} + b = \pm 1$ (canonical form).
4. Margin width $= 2/\|\mathbf{w}\|$. Maximise margin = minimise $\frac{1}{2}\|\mathbf{w}\|^2$.
5. Constraint: $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \; \forall i$.

Hard margin: $\min \frac{1}{2}\|\mathbf{w}\|^2$ s.t. $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$.
Soft margin: $\min \frac{1}{2}\|\mathbf{w}\|^2 + C\sum \xi_i$ s.t. $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$.

Example 8.1b — 2D SVM hard-margin (manual)

Two positives at $\mathbf{x}_+ = (1, 0), (0, 1)$ and two negatives at $\mathbf{x}_- = (-1, 0), (0, -1)$.
By symmetry the boundary passes through origin. Try $\mathbf{w} = (1, 1), b = 0$: scaled so the closest pos point has $\mathbf{w}^\top \mathbf{x} + b = 1$. $\mathbf{w}^\top(1, 0) = 1$ ✓. Check negative: $\mathbf{w}^\top(-1, 0) = -1$ ✓.
Margin: $2/\|\mathbf{w}\| = 2/\sqrt{2} = \sqrt{2}$.
Support vectors: all four points (they all sit on the margin).

8.2. Support vectors on a 1D dataset

Example 8.1 — $\{(1, +), (2, +), (3, -), (4, -)\}$

Closest opposing points: $x = 2$ (+) and $x = 3$ (−). Decision: $f(x) = \text{sign}(2.5 - x)$, i.e. + if $x < 2.5$.
Support vectors: $\{2, 3\}$.

8.3. Kernel trick

$\kappa(\mathbf{p}, \mathbf{q}) = \phi(\mathbf{p})^\top \phi(\mathbf{q})$. Replace dot products in the dual with κ.

Example 8.0 — build a kernel matrix and check PSD numerically

Three 1D points $\{1, 2, 3\}$, polynomial kernel $\kappa(p, q) = (pq + 1)^2$.
$K_{ij} = (p_i p_j + 1)^2$:

$$K = \begin{bmatrix} (1+1)^2 & (2+1)^2 & (3+1)^2 \\ (2+1)^2 & (4+1)^2 & (6+1)^2 \\ (4+1)^2 & (6+1)^2 & (9+1)^2 \end{bmatrix} = \begin{bmatrix} 4 & 9 & 16 \\ 9 & 25 & 49 \\ 16 & 49 & 100 \end{bmatrix}.$$

Symmetric? $K = K^\top$ — yes, by construction.
PSD check (eigenvalues): $\mathrm{tr}(K) = 129$, $\det(K)$ tedious; the cheap check: K is a Gram matrix of $\phi(\mathbf{x}_i) = [\mathbf{x}_i^2, \sqrt{2}\mathbf{x}_i, 1]^\top$, so $K = \Phi\Phi^\top$, automatically PSD.

8.4. Proving K is symmetric and PSD (recurring 7.5-marker)

Recipe

Symmetric: $K_{ij} = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$. Dot product is commutative, so $K_{ij} = K_{ji} \Rightarrow K = K^\top$.
PSD: For any $\mathbf{c} \in \mathbb{R}^N$:

$$\mathbf{c}^\top K \mathbf{c} = \sum_{i,j} c_i c_j \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) = \left\| \textstyle\sum_i c_i \phi(\mathbf{x}_i) \right\|^2 \geq 0.$$

Equivalently, with Φ having rows $\phi(\mathbf{x}_i)^\top$, $K = \Phi\Phi^\top \Rightarrow \mathbf{c}^\top K \mathbf{c} = \|\Phi^\top \mathbf{c}\|^2 \geq 0$.

8.5. Finding ϕ for $\kappa(\mathbf{p}, \mathbf{q}) = (\mathbf{p}^\top \mathbf{q})^d$

Example 8.2 — $d = 3$, 2D inputs

$\mathbf{p} = (p_1, p_2), \mathbf{q} = (q_1, q_2)$. Expand $(\mathbf{p}^\top \mathbf{q})^3 = (p_1 q_1 + p_2 q_2)^3 = p_1^3 q_1^3 + 3p_1^2 p_2 q_1^2 q_2 + 3p_1 p_2^2 q_1 q_2^2 + p_2^3 q_2^3$.
Match with $\phi(\mathbf{p})^\top \phi(\mathbf{q})$:

$$\phi(\mathbf{p}) = \begin{bmatrix} p_1^3 \\ \sqrt{3}\, p_1^2 p_2 \\ \sqrt{3}\, p_1 p_2^2 \\ p_2^3 \end{bmatrix}.$$

Pattern — $(\mathbf{p}^\top \mathbf{q} + c)^d$

Use binomial expansion. The $\sqrt{\binom{d}{k}}$ coefficients are absorbed into ϕ.

9. Topic 7 — Clustering: K-Means and GMM

9.1. K-Means iterations recipe

Recipe

1. Initialise: assign each point to a cluster (or pick K centroids).
2. Compute centroids: $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} \mathbf{x}$.
3. Reassign: $C(\mathbf{x}_i) = \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$.
4. Repeat 2–3 until no reassignment.

Empty cluster handling: re-seed with the point farthest from its centroid, or drop K by 1.

9.2. Worked example (10 points, 3 clusters — MidSem Q3)

For $\{[1,1,1],[-1,-1,-1],[0,0,0],[5,5,0],[4,4,0],[6,6,0],[6,4,0],[4,6,0],[0,-6,-4],[0,-4,-6]\}$ with init $C_1=\{1,4,7,10\}, C_2=\{2,5,8\}, C_3=\{3,6,9\}$:

- **Iteration 1 centroids:** $\boldsymbol{\mu}_1=[3,1.5,-1.25]$, $\boldsymbol{\mu}_2=[2.33,3,-0.33]$, $\boldsymbol{\mu}_3=[2,0,-1.33]$.
- Reassign: $C_1=\{\}$ (*empty!*), $C_2=\{4,5,6,7,8\}$, $C_3=\{1,2,3,9,10\}$. Re-seed $\boldsymbol{\mu}_1$ with the farthest point (point 9).
- **Iteration 2:** $\boldsymbol{\mu}_1=[0,-6,-4]$, $\boldsymbol{\mu}_2=[5,5,0]$, $\boldsymbol{\mu}_3=[0,-1,-1.5]$. Reassign $\rightarrow C_1=\{9,10\}, C_2=\{4,5,6,7,8\}, C_3=\{1,2,3\}$.
- **Iteration 3:** $\boldsymbol{\mu}_1=[0,-5,-5], \boldsymbol{\mu}_2=[5,5,0], \boldsymbol{\mu}_3=[0,0,0]$. No change $\Rightarrow$ *converged*.

9.3. K-Means is non-convex

The objective involves discrete assignments (NP-hard in general). Multiple restarts (e.g. K-Means++) help. Final assignment depends on initialisation.

9.4. GMM and EM (concept-level)

GMM models $p(\mathbf{x})=\sum_k \pi_k \mathcal{N}(\mathbf{x};\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)$. EM alternates:

- **E-step:** compute responsibilities $\gamma_{ik}=\pi_k\mathcal{N}(\mathbf{x}_i;\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)/\sum_j \pi_j\mathcal{N}(\mathbf{x}_i;\boldsymbol{\mu}_j,\boldsymbol{\Sigma}_j)$.
- **M-step:** update $\boldsymbol{\mu}_k=\sum_i \gamma_{ik}\mathbf{x}_i/\sum_i \gamma_{ik}$, similarly $\boldsymbol{\Sigma}_k, \pi_k$.

K-Means is the hard-assignment, isotropic-covariance limit of EM on GMM.

Example 9.1 — one E-step on a 1D 2-component GMM

Data $\{1,2,8,9\}$, init $\mu_1=1,\mu_2=9,\sigma_1=\sigma_2=1,\pi_1=\pi_2=0.5$.
For point $\mathbf{x}=2$: $\mathcal{N}(2;1,1)=\frac{1}{\sqrt{2\pi}}e^{-0.5}\approx 0.242$; $\mathcal{N}(2;9,1)=\frac{1}{\sqrt{2\pi}}e^{-24.5}\approx 10^{-11}$.
Responsibilities: $\gamma_1\approx 1,\gamma_2\approx 0$ — the point is essentially assigned to cluster 1 (just like K-Means would).
For point $\mathbf{x}=8$ symmetrically: $\gamma_1\approx 0,\gamma_2\approx 1$.
M-step: $\mu_1=(1\cdot 1+1\cdot 2)/2=1.5,\ \mu_2=8.5$. Iterate until convergence.

10. Topic 8 — Practical ML: Splits, Metrics, Normalisation

10.1. Confusion matrix

	Pred +	Pred −
Actual +	TP	FN
Actual −	FP	TN

- Accuracy $=(TP+TN)/N$.
- Precision $=TP/(TP+FP)$ — "of those flagged +, how many were really +?"
- Recall (sensitivity) $=TP/(TP+FN)$ — "of all actual +, how many did we catch?"
- F1 $=2\cdot\frac{P\cdot R}{P+R}$.

10.2. Worked threshold sweep (recurring)

Example 10.1

Scores $0.7, 0.65, 0.92, 0.75, 0.45, 0.8, 0.3, 0.8, 0.1, 0.1$;
labels $+, -, +, +, -, +, +, +, -, -$. Threshold $\theta = 0.5$.
Predictions (≥ 0.5): $+, +, +, +, -, +, -, +, -, -$.
TP=5 (idx 1,3,4,6,8), FP=1 (idx 2), FN=1 (idx 7), TN=3.
Precision $= 5/6 \approx 0.83$, Recall $= 5/6 \approx 0.83$.
At $\theta = 0.2$: TP=6, FP=2, FN=0 $\Rightarrow$ Precision $= 6/8 = 0.75$, Recall $= 6/6 = 1$.

10.3. Normalisation

- **Mean-centering:** $\mathbf{x} \leftarrow \mathbf{x} - \boldsymbol{\mu}$.
- **Z-score:** $z_i = (x_i - \mu)/\sigma$. Pop or sample std — both accepted.
- **Min-max:** $x \leftarrow (x - \min)/(\max - \min) \in [0, 1]$.
- **L2 unit-norm per sample:** $\mathbf{x} \leftarrow \mathbf{x}/\|\mathbf{x}\|_2$.

10.4. Cross-validation

k-fold CV: split into k folds, train on $k-1$, validate on the held-out fold; average. Use stratified k-fold for imbalanced classification. **Leave-one-out** $= k = N$ (expensive).

Example 10.2 — 3-fold CV on 6 samples

Samples labelled $\{1, 2, 3, 4, 5, 6\}$, 3 folds:
- Fold 1: train $\{3, 4, 5, 6\}$, validate $\{1, 2\} \rightarrow$ acc a_1.
- Fold 2: train $\{1, 2, 5, 6\}$, validate $\{3, 4\} \rightarrow$ acc a_2.
- Fold 3: train $\{1, 2, 3, 4\}$, validate $\{5, 6\} \rightarrow$ acc a_3.

Reported metric: mean $\bar{a} = (a_1 + a_2 + a_3)/3$, std σ_a.
Don't peek at the test set during CV — it stays untouched until final evaluation.

10.5. Hyperparameter tuning

Grid search vs random search vs Bayesian. *Always* on the *validation* fold; test set is touched only once.

11. Topic 9 — Perceptron and the Single-Layer Classifier

11.1. Algorithm

Perceptron training

Initialise $\mathbf{w}, b$. For each example $(\mathbf{x}_i, y_i)$ with $y_i \in \{-1, +1\}$:
1. $\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x}_i + b)$.
2. If $\hat{y} \neq y_i$ (misclassified): $\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$, $b \leftarrow b + \eta y_i$.
3. If correct: *no update*.

Repeat over the dataset until convergence (linearly separable data) or max epochs.

11.2. Worked example (Quiz 2 Variant 3)

Example 11.1 — three points, $\eta = 0.5$

$D = \{([1,1],+1),([0,0],+1),([-1,-1],-1)\}$. Init $w_0 = 0, w_1 = -0.8, w_2 = -0.8$.
Iteration 1:

- $([1,1],+1)$: $a = 0 - 0.8 - 0.8 = -1.6 \Rightarrow \hat{y} = -1 \neq +1$. Update: $w_0 = 0 + 0.5 = 0.5$, $w_1 = -0.8 + 0.5 = -0.3$, $w_2 = -0.3$.
- $([0,0],+1)$: $a = 0.5 \geq 0 \Rightarrow \hat{y} = +1$. *No update.*
- $([-1,-1],-1)$: $a = 0.5 + 0.3 + 0.3 = 1.1 \Rightarrow \hat{y} = +1 \neq -1$. Update: $w_0 = 0.5 - 0.5 = 0$, $w_1 = -0.3 + 0.5 = 0.2$, $w_2 = 0.2$.

End of epoch 1: $w = [0, 0.2, 0.2]$.
Iteration 2: all three classified correctly $\Rightarrow$ converged.

Example 11.2 — perceptron with bias augmentation (PYQ style)

Class A: $\mathbf{x}_1 = [1,2]^\top, \mathbf{x}_2 = [4,2]^\top$ (label $+1$). Class B: $\mathbf{x}_3 = [3,0]^\top$ (label -1). Init $\mathbf{w}^0 = [1,1,0]^\top$, $\eta = 0.1$.
Augment with bias=1: $\tilde{\mathbf{x}}_1 = [1,2,1]^\top, \tilde{\mathbf{x}}_2 = [4,2,1]^\top, \tilde{\mathbf{x}}_3 = [3,0,1]^\top$.
Iteration 1:

- $\tilde{\mathbf{x}}_1$: $\mathbf{w}^\top \tilde{\mathbf{x}} = 1 \cdot 1 + 1 \cdot 2 + 0 \cdot 1 = 3 > 0 \rightarrow +1$ ✓. No update.
- $\tilde{\mathbf{x}}_2$: $1 \cdot 4 + 1 \cdot 2 + 0 = 6 > 0 \rightarrow +1$ ✓. No update.
- $\tilde{\mathbf{x}}_3$: $1 \cdot 3 + 1 \cdot 0 + 0 = 3 > 0 \rightarrow +1$, but $y = -1 \Rightarrow$ **misclassified**. Update $\mathbf{w} \leftarrow \mathbf{w} + \eta y \tilde{\mathbf{x}}_3 = [1,1,0] + 0.1(-1)[3,0,1] = [0.7, 1, -0.1]^\top$.

End of iteration 1: $\mathbf{w}^1 = [0.7, 1, -0.1]^\top$ (i.e. $w_1 = 0.7, w_2 = 1, b = -0.1$).

11.3. XOR — why a single perceptron fails

$\{(0,0) \rightarrow 0, (0,1) \rightarrow 1, (1,0) \rightarrow 1, (1,1) \rightarrow 0\}$ is not linearly separable. Need at least one hidden layer (2 neurons + 1 output).

11.4. Connecting perceptron to logical gates (Quiz 2 PYQ)

A perceptron with $w_0 = 1, w_1 = -1, w_2 = -1$ and ± 1 logic computes $x = 1 - x_1 - x_2$ then $\phi(x) = \text{sign}(x)$. Truth table:

x_1	x_2	$1 - x_1 - x_2$	ϕ
$+1$	$+1$	-1	-1
$+1$	-1	1	$+1$
-1	$+1$	1	$+1$
-1	-1	3	$+1$

This is **NAND** (in ± 1 logic). Memorise: $w_0 = 1, w_1 = -1, w_2 = -1 \Rightarrow$ NAND.

12. Topic 10 — MLP, Backprop, Initialisation

12.1. Standard MLP forward pass (2-2-1)

$$p_j = \sum_i w_{ji} x_i, \quad q_j = \phi_1(p_j), \quad s = \sum_j u_j q_j, \quad o = \phi_2(s), \quad L = \tfrac{1}{2}(y - o)^2.$$

12.2. Chain rule for $\partial L / \partial w_{ij}$

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial q_j} \cdot \frac{\partial q_j}{\partial p_j} \cdot \frac{\partial p_j}{\partial w_{ij}} = (o - y) \cdot \phi_2'(s) \cdot u_j \cdot \phi_1'(p_j) \cdot x_i.$$

For ReLU: $\phi_1'(p) = \mathbf{1}[p > 0]$, undefined exactly at 0 (use 0 or 1 by convention).

12.3. Worked example (Quiz 2 Variant 1)

Example 12.1

$W = \begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}$, $\mathbf{u} = \begin{bmatrix}2\\ 1\end{bmatrix}$, $\mathbf{x} = \begin{bmatrix}1\\ 1\end{bmatrix}$, $y = 2$.
$p_1 = p_2 = 2$, $q_1 = q_2 = 2$, $s = 2 \cdot 2 + 1 \cdot 2 = 6$, $o = 6$. Loss $= \frac{1}{2}(2-6)^2 = 8$.
$\partial L/\partial o = o - y = 4$. $p_2 > 0 \Rightarrow$ ReLU' $= 1$. $\partial L/\partial w_{22} = 4 \cdot 1 \cdot u_2 \cdot 1 \cdot x_2 = 4 \cdot 1 \cdot 1 \cdot 1 = \boxed{4}$.

12.4. Why *not* init weights all equal (e.g. all 1.0)

Symmetry-breaking failure. All hidden units in a layer get the same input, the same activation, the same gradient — they remain identical forever. The network has the expressive capacity of a single neuron per layer.
Fix: small random init (e.g. Xavier/Glorot or He). Zeros are also bad for the same reason, plus dead gradients.

12.5. Why don't we just set $\partial L/\partial w_{ij} = 0$ and solve?

For a non-linear deep network, this is a system of *nonlinear* equations in millions of variables. There is no closed-form solver. Backprop = gradient descent is computationally tractable.

12.6. If ϕ_1 is linear, MLP collapses to SLP

$o = \mathbf{u}^\top \mathbf{W}\mathbf{x} = (\mathbf{W}^\top \mathbf{u})^\top \mathbf{x}$. Compute $\mathbf{v} = \mathbf{W}^\top \mathbf{u}$ and the SLP weights are $v_1, \ldots, v_d$.

Example 12.2

$W = \begin{bmatrix}-1 & 1\\ -2 & 1\end{bmatrix}, \mathbf{u} = \begin{bmatrix}2\\ 2\end{bmatrix} \Rightarrow \mathbf{v} = \mathbf{W}^\top \mathbf{u} = \begin{bmatrix}-1 & -2\\ 1 & 1\end{bmatrix}\begin{bmatrix}2\\ 2\end{bmatrix} = \begin{bmatrix}-6\\ 4\end{bmatrix}$.
Equivalent SLP: $o = -6x_1 + 4x_2$.

12.7. Activations cheat-sheet

Activation	Formula	Derivative	Use
Sigmoid	$\sigma(z) = 1/(1+e^{-z})$	$\sigma(z)(1-\sigma(z))$	Binary out
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	$1 - \tanh^2(z)$	Hidden (rare)
ReLU	$\max(0, z)$	$\mathbf{1}[z > 0]$	Hidden default
LeakyReLU	$\max(\alpha z, z)$	α for $z < 0$, else 1	Avoids dead ReLU
Softmax	$e^{z_i} / \sum_j e^{z_j}$	matrix Jacobian	Multi-class out
GELU	$z\Phi(z)$	—	Transformers

Example 12.3 — sigmoid and tanh derivatives at a point

At $z = 1$: $\sigma(1) = 1/(1+e^{-1}) \approx 0.731$; $\sigma'(1) = 0.731 \cdot (1 - 0.731) \approx 0.197$.
$\tanh(1) \approx 0.762$; $\tanh'(1) = 1 - 0.762^2 \approx 0.420$.
At $z = 5$: $\sigma'(5) \approx 0.0066$ — *tiny gradient*, this is exactly the vanishing-gradient problem when many sigmoids stack.

Example 12.4 — Xavier vs He initialisation

A FC layer with $n_{\text{in}} = 100, n_{\text{out}} = 50$.
Xavier (sigmoid/tanh): sample weights from $\mathcal{N}(0,\, 2/(n_{\text{in}} + n_{\text{out}})) = \mathcal{N}(0,\, 2/150)$, std

≈ 0.115.
He (ReLU): sample from $\mathcal{N}(0,\, 2/n_{\text{in}}) = \mathcal{N}(0,\, 0.02)$, std ≈ 0.141.
Why scale by $1/\sqrt{n_{\text{in}}}$? Variance of $\mathbf{w}^\top \mathbf{x}$ is $n_{\text{in}} \cdot \text{Var}(w) \cdot \text{Var}(x)$; to keep activations on the same scale across layers, $\text{Var}(w) \propto 1/n_{\text{in}}$.

12.8. Universal Approximation Theorem (concept)

A feed-forward network with one hidden layer and a non-linear activation can approximate any continuous function on a compact domain — *given enough hidden units*. **Caveat:** "can" doesn't mean "is easy to train"; depth and structure matter.

13. Topic 11 — Optimisation: GD, Momentum, Newton, Adam

13.1. Standard GD update equations (must memorise)

Plain GD:	$\mathbf{w}^{k+1} = \mathbf{w}^k - \eta \nabla L(\mathbf{w}^k)$
Momentum:	$\mathbf{v}^{k+1} = \beta \mathbf{v}^k + \nabla L(\mathbf{w}^k);\ \mathbf{w}^{k+1} = \mathbf{w}^k - \eta \mathbf{v}^{k+1}$
Newton:	$\mathbf{w}^{k+1} = \mathbf{w}^k - [H^k]^{-1} \nabla L(\mathbf{w}^k)$
RMSProp:	$v \leftarrow \beta v + (1-\beta) g^2;\ \mathbf{w} \leftarrow \mathbf{w} - \eta g / \sqrt{v + \varepsilon}$
Adam:	moments m, v on g and g^2 + bias correction

13.2. The "one-step convergence" trick

For a quadratic $f(w) = aw^2 + bw + c$, $f''(w) = 2a$, Newton's step:

$$\eta^* = \frac{1}{f''(w)} = \frac{1}{2a}.$$

This converges in *exactly one step* regardless of starting point. Always answer this.

13.3. Worked example (Quiz 1 / PYQ)

Example 13.1 — $\mathcal{L}(w) = 2w^2 - 4w + 7,\ w^0 = 2,\ \eta = 0.2$

$\mathcal{L}'(w) = 4w - 4$.
$w^1 = 2 - 0.2(4 \cdot 2 - 4) = 2 - 0.2 \cdot 4 = 1.2$.
$w^2 = 1.2 - 0.2(4 \cdot 1.2 - 4) = 1.2 - 0.16 = 1.04$.
Optimum at $w^* = 1$, $\mathcal{L}(1) = 5$.
One-step η: $1/(2a) = 1/4 = 0.25$.

Example 13.2 — momentum vs plain GD on $f(w) = 2w^2 - 4w + 7$

$f'(w) = 4w - 4$. Start $w^0 = 2, \eta = 0.2$.
Plain GD: $w^1 = 1.2$, $w^2 = 1.04$ (already shown).
Momentum, $\beta = 0.9$, $v^0 = 0$:
$v^1 = 0.9 \cdot 0 + f'(2) = 4$. $w^1 = 2 - 0.2 \cdot 4 = 1.2$. (Same as plain GD on step 1 since $v^0 = 0$.)
$v^2 = 0.9 \cdot 4 + f'(1.2) = 3.6 + 0.8 = 4.4$. $w^2 = 1.2 - 0.2 \cdot 4.4 = 0.32$ — *overshoots* the optimum $w^* = 1$ on the other side.
$v^3 = 0.9 \cdot 4.4 + f'(0.32) = 3.96 + (-2.72) = 1.24$. $w^3 = 0.32 - 0.2 \cdot 1.24 = 0.072$ — still overshooting.
Lesson: momentum accelerates but can overshoot near the optimum; combine with smaller η near convergence (LR scheduling).

13.4. Momentum and local minima — careful wording

"Adding a momentum term *removes* the problem of local minima" is **FALSE**. The right phrasing is "*reduces*" (it can carry the iterate over shallow minima but is not a guarantee).

13.5. Adam intuition (PYQ-friendly)

Adam = momentum (first moment) + RMSProp (second moment) + bias correction. Per-parameter adaptive learning rates; very robust default choice ($\eta = 10^{-3}, \beta_1 = 0.9, \beta_2 = 0.999$).

Example 13.3 — Newton vs GD on a 2D quadratic

$f(\mathbf{w}) = w_1^2 + 10w_2^2$, $\nabla f = [2w_1, 20w_2]^\top$, Hessian $H = \begin{bmatrix} 2 & 0 \\ 0 & 20 \end{bmatrix}$. Start $\mathbf{w}^0 = [1, 1]$.

Plain GD with $\eta = 0.05$: $\mathbf{w}^1 = [1 - 0.1,\ 1 - 1] = [0.9, 0]$ — overshoots in w_2 direction. η has to be small ($< 2/20 = 0.1$) to be stable in w_2 but then w_1 converges slowly.
Newton: $\mathbf{w}^1 = \mathbf{w}^0 - H^{-1}\nabla f = [1, 1] - [1, 1] = [0, 0]$ — *exact* optimum in one step. The condition number of H doesn't matter.

13.6. Vanishing / exploding gradients

- Many sigmoid/tanh layers: gradients shrink toward zero through the chain rule.
- ReLU avoids saturation in the positive region.
- Residual / skip connections: $h(x) = f(x) + x$ — the gradient has an *identity highway* $\partial L/\partial x = \partial L/\partial h \cdot (1 + f'(x))$. Mitigates vanishing in deep networks.
- Batch norm and careful weight init (Xavier/He) also help.

Example 13.4 — vanishing gradient through 5 sigmoid layers

Suppose every layer's local derivative is $\sigma' \approx 0.25$ (its maximum) and weights are 1.
Gradient through 5 layers: $0.25^5 \approx 0.00098$ — three orders of magnitude smaller than at the output.
With a residual connection: $\partial L/\partial x = \partial L/\partial h \cdot (1 + f'(x)) \geq \partial L/\partial h$. The identity branch keeps the gradient *above* 1.

13.7. Stochastic minibatch GD pseudocode (PYQ Q3)

Pseudocode — SGD on $L = \sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$

```
Inputs: D = {(x_i, y_i)}, learning rate eta, batch size m, epochs E
Output: optimal w

1: Initialize w (small random / zeros)
2: for epoch = 1..E do
3:    Shuffle D
4:    for j = 0..floor((N-1)/m) do
5:       B_j = batch j of size m_j
6:       grad = -2/m_j * sum_{(x,y) in B_j} (y - w.T x) * x
7:       w = w - eta * grad
8: return w
```

Word answers: (i) optimal **w**; (ii) random / zeros; (iii) yes (this loss is convex in **w**); (iv) no on convex problems with appropriate η; (v) monitor loss, tune η, normalise inputs, gradient checking, shuffle each epoch; (vi) stop on small change in loss / early stopping / small gradient

norm.

14. Topic 12 — Regularisation & NN Training Tricks

14.1. Dropout

Randomly zero each activation with probability p during training; scale by $1/(1-p)$ at test time (or scale during training — "inverted dropout"). Acts like an ensemble of sub-networks. **Key:** reduces *overfitting* by adding stochasticity. Does *not* compress the model; inference uses all weights.

Example 14.1 — inverted dropout with $p = 0.5$

Layer activations $\mathbf{a} = [2, 4, 6, 8]$. Random mask $\mathbf{m} = [1, 0, 1, 0]$ (each entry Bernoulli(0.5)).
During training: $\mathbf{a}' = \mathbf{a} \odot \mathbf{m}/(1-p) = [2, 0, 6, 0]/0.5 = [4, 0, 12, 0]$. The kept activations are scaled up so that the *expected* sum equals the no-dropout sum.
At test time: no mask, no scaling — use full $[2, 4, 6, 8]$.
Vanishing the model: setting $p = 0$ disables dropout entirely; $p = 1$ kills the layer.

14.2. Batch Normalisation

Per-layer, per-batch:

$$\hat{x} = (x - \mu_B)/\sqrt{\sigma_B^2 + \varepsilon}, \quad y = \gamma\hat{x} + \beta.$$

Effects:

- Stabilises and accelerates training; allows higher η.
- Acts as a mild regulariser (mini-batch noise).
- At test time, use running averages of μ_B, σ_B^2.

Example 14.0 — BatchNorm on a mini-batch

Activations of one neuron over a batch of 4: $x = \{1, 3, 5, 7\}$. $\gamma = 2, \beta = 1, \varepsilon = 10^{-5}$.
$\mu_B = 4$, $\sigma_B^2 = \frac{1}{4}((1-4)^2 + (3-4)^2 + (5-4)^2 + (7-4)^2) = \frac{1}{4}(9+1+1+9) = 5$, $\sigma_B \approx 2.236$.
$\hat{x} = (x-4)/2.236 = \{-1.342, -0.447, 0.447, 1.342\}$.
Output $y = \gamma\hat{x} + \beta = \{1 - 2.683, 1 - 0.894, 1 + 0.894, 1 + 2.683\} = \{-1.683, 0.106, 1.894, 3.683\}$.

14.3. Layer Norm vs Batch Norm

Batch norm: across the batch dimension (good for CNNs). Layer norm: across feature dimension within a single sample (good for sequences/transformers).

14.4. Weight decay

Equivalent to L_2 regularisation: $\mathbf{w} \leftarrow \mathbf{w} - \eta(\nabla L + \lambda\mathbf{w})$. Keeps weights small.

14.5. Other tricks

Early stopping, data augmentation, label smoothing, learning-rate scheduling (step / cosine / warm restarts), gradient clipping (for RNNs).

15. Topic 13 — Convolutional Neural Networks

15.1. Why convolution?

- **Local connectivity:** a neuron sees only a small receptive field.
- **Weight sharing:** the same kernel slides over the whole input, so far fewer params than FC.
- **Translation equivariance:** a feature shifted in the input shifts equally in the output.

15.2. Conv arithmetic

Output spatial size (1D): $\lfloor (L + 2P - k)/s \rfloor + 1$. 2D version applied independently to height and width.

Parameters in a conv layer:

$$\underbrace{k_h \cdot k_w \cdot C_{\text{in}} \cdot C_{\text{out}}}_{\text{weights}} + \underbrace{C_{\text{out}}}_{\text{biases (if any)}} .$$

15.3. Worked example — 1D conv (PYQ)

Example 14.1 — $C_{\text{in}} = 3, L = 1000, k = 7, C_{\text{out}} = 5, s = 3$, no bias

Params: $7 \cdot 3 \cdot 5 = \boxed{105}$ (with bias: 110).

Example 14.2 — PyTorch-style `nn.Conv1d(1,5,7,padding=3,bias=True)`, input $(12, 1, 100)$

Params: $7 \cdot 1 \cdot 5 + 5 = \boxed{40}$.
Output: $(12,\ 5,\ \lfloor (100 + 6 - 7)/1 \rfloor + 1) = (12, 5, 100)$.

15.4. Pooling

Max-pool / avg-pool reduce spatial dims; *no learnable params*. They make features *translation-invariant* (small shifts don't change the pooled output).

15.5. Total trainable params in a tiny model (PYQ)

Example 14.3

```
fc1 = nn.Linear(10, 4, bias=False)    # 10*4 = 40
fc2 = nn.Linear(4, 2, bias=True)      # 4*2 + 2 = 10
```

Total: $40 + 10 = \boxed{50}$.

15.6. Landmark CNN architectures (one-line each)

- **LeNet-5** (1998): 2 conv + 2 pool + 2 FC. Digits.
- **AlexNet** (2012): ReLU, dropout, GPU; ImageNet breakthrough.
- **VGG** (2014): stack of 3×3 convs; deep but uniform.
- **GoogLeNet/Inception**: parallel branches at multiple scales; 1×1 convs for dim reduction.
- **ResNet** (2015): residual connections enable 100+ layers.
- **DenseNet**: every layer connects to all subsequent.

- **MobileNet/EfficientNet**: depthwise-separable convs, scaling laws.

15.7. Receptive field

Stack of L convs (kernel k, stride 1, no dilation): receptive field grows $1 + L(k-1)$ in each dim. Two 3×3 convs = effective 5×5 but with fewer params ($2 \cdot 9 = 18$ vs 25).

Example 15.1 — receptive field in a 3-conv stack

Three conv layers, kernel 3×3, stride 1, no dilation: receptive field $= 1 + 3(3-1) = 7 \times 7$. Same effective view as one 7×7 conv but params: $3 \cdot 9 = 27$ vs 49 — and the deeper net has more nonlinearity.

Example 15.2 — Conv2D output and params

Input $32 \times 32 \times 3$, layer: $C_{\text{out}} = 16$, $k = 5$, $P = 2$, $s = 1$, bias yes.
Output spatial: $\lfloor (32 + 4 - 5)/1 \rfloor + 1 = 32$. Output: $32 \times 32 \times 16$.
Params: $5 \cdot 5 \cdot 3 \cdot 16 + 16 = 1216$.

16. Topic 14 — RNNs and LSTMs

16.1. Vanilla RNN

$$\mathbf{h}_t = \phi(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b}), \quad \mathbf{y}_t = W_y \mathbf{h}_t.$$

Same W_x, W_h, W_y at every time step (weight sharing).

16.2. Backprop through time (BPTT)

Unroll the RNN over T steps and backpropagate. Gradient through $\mathbf{h}_T$ to $\mathbf{h}_0$ involves the product $\prod_t W_h^\top \operatorname{diag}(\phi'(\cdot))$ — eigenvalues < 1 vanish, > 1 explode.

16.3. LSTM gates (memorise the four)

Forget f_t, input i_t, candidate $\tilde{C}_t$, output o_t:

$$\begin{aligned}
f_t &= \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \\
i_t &= \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \\
\tilde{C}_t &= \tanh(W_C[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \\
C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \\
o_t &= \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \\
\mathbf{h}_t &= o_t \odot \tanh(C_t)
\end{aligned}$$

The cell state C_t has an *additive* update — the gradient highway that mitigates vanishing.

Example 16.0 — RNN forward pass over 3 time steps

$\mathbf{x}_1 = 1, \mathbf{x}_2 = 2, \mathbf{x}_3 = -1$. $W_x = 0.5, W_h = 0.4, W_y = 1$, $\mathbf{b} = 0$, $\phi = \tanh$, $\mathbf{h}_0 = 0$.
$\mathbf{h}_1 = \tanh(0.5 \cdot 1 + 0.4 \cdot 0) = \tanh(0.5) \approx 0.462$.
$\mathbf{h}_2 = \tanh(0.5 \cdot 2 + 0.4 \cdot 0.462) = \tanh(1.185) \approx 0.829$.
$\mathbf{h}_3 = \tanh(0.5 \cdot (-1) + 0.4 \cdot 0.829) = \tanh(-0.169) \approx -0.167$.
Outputs: $\mathbf{y}_t = W_y \mathbf{h}_t = \{0.462, 0.829, -0.167\}$.
Watch: the same W_x, W_h, W_y are reused at every step — that's the whole point.

Example 16.1 — LSTM cell update (sketch)

At time t: $\mathbf{h}_{t-1}, C_{t-1}$ given. Concatenate $[\mathbf{h}_{t-1}, \mathbf{x}_t]$, apply four linear layers + activations.
Suppose all gates output 0.7 (high) and $\tilde{C}_t = 0.5$, $C_{t-1} = 2$.
$C_t = 0.7 \cdot 2 + 0.7 \cdot 0.5 = 1.4 + 0.35 = 1.75$.
$\mathbf{h}_t = 0.7 \cdot \tanh(1.75) \approx 0.7 \cdot 0.941 = 0.659$.
Key: C_t depends *additively* on C_{t-1} — this is the gradient highway that vanilla RNNs lack.

16.4. Applications

Language modelling, machine translation (encoder-decoder before transformers), speech recognition, time-series forecasting.

17. Topic 15 — Transformers and Self-Attention

17.1. Word embeddings

Sparse one-hot → dense $\mathbb{R}^d$ via an embedding matrix. Word2Vec / GloVe / contextual (BERT).

17.2. Self-attention (one head)

Given inputs $X \in \mathbb{R}^{n \times d}$:

$$Q = XW_Q,\ K = XW_K,\ V = XW_V, \quad \text{Attn}(X) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Each token attends to every other token — captures long-range dependencies in $O(1)$ steps.

17.3. Multi-head attention

Stack h heads with their own $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)} \in \mathbb{R}^{d \times d_h}$ (with $d_h = d/h$). Concatenate the h outputs $(n \times d)$, project with $W_O \in \mathbb{R}^{d \times d}$.

Dimensionality (PYQ-style)

If $\mathbf{x}_i \in \mathbb{R}^d$ and there are h heads, then per head: W_Q, W_K, W_V are $d \times d_h$; W_O is $h \cdot d_h \times d_o$ (often $d_o = d$).
For a 2-head, 2D-input, 2D-output network with each head W being square: all six $W^{(i)}$ are 2×2, W_O is 2×4.

17.4. Worked example — multi-head, no softmax (PYQ EndSem)

Example 16.1

Inputs: $\mathbf{x}_1 = [1,1], \mathbf{x}_2 = [-1,-1], \mathbf{x}_3 = [2,2], \mathbf{x}_4 = [0,0]$. Head 1: $W_Q = W_K = W_V = I$. Head 2: $W_Q = J$ (all 1s), $W_K = -J$, $W_V = I$. W_O all 1s.
Head 1 (no softmax): $y_i^{(1)} = \sum_j (\mathbf{x}_i \cdot \mathbf{x}_j)\mathbf{x}_j$.
$\mathbf{x}_1 \cdot \mathbf{x}_1 = 2, \mathbf{x}_1 \cdot \mathbf{x}_2 = -2, \mathbf{x}_1 \cdot \mathbf{x}_3 = 4, \mathbf{x}_1 \cdot \mathbf{x}_4 = 0$.
$y_1^{(1)} = 2[1,1] - 2[-1,-1] + 4[2,2] + 0 = [12,12]$. (similarly: $y_2^{(1)} = [-12,-12], y_3^{(1)} = [24,24], y_4^{(1)} = [0,0]$.)
Head 2: $Q_i^{(2)} = J\mathbf{x}_i = (\mathbf{1}^\top \mathbf{x}_i)[1,1]^\top$. Let $s_j = \mathbf{1}^\top \mathbf{x}_j$.
$Q_i^{(2)} \cdot K_j^{(2)} = -2s_i s_j$. So $y_i^{(2)} = -2s_i \sum_j s_j \mathbf{x}_j$.
$s_1 = 2, s_2 = -2, s_3 = 4, s_4 = 0 \Rightarrow \sum s_j \mathbf{x}_j = 2[1,1] - 2[-1,-1] + 4[2,2] = [12,12]$
... actually: $2 \cdot [1,1] + (-2) \cdot [-1,-1] + 4 \cdot [2,2] + 0 = [2,2] + [2,2] + [8,8] = [12,12]$.

$y_1^{(2)} = -2 \cdot 2 \cdot [12, 12] = [-48, -48]$, similarly $[48, 48], [-96, -96], [0, 0]$.
Concat & output: W_O all 1s sums all four entries.
$y_1 = 12 + 12 + (-48) + (-48) = [-72, -72]$, etc.

Example 17.0 — single-head attention numerical

Two tokens, $d_k = 2$. $X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. Take $W_Q = W_K = W_V = I$.
$Q = K = V = X$.
$QK^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. Divide by $\sqrt{d_k} = \sqrt{2}$: $\begin{bmatrix} 0.707 & 0 \\ 0 & 0.707 \end{bmatrix}$.
softmax row-wise: row $1 = [e^{0.707}, e^0]/(e^{0.707} + 1) = [2.028, 1]/3.028 = [0.670, 0.330]$.
Row $2 = [0.330, 0.670]$ symmetrically.
Output $= \text{softmax}(\cdot)V = \begin{bmatrix} 0.670 & 0.330 \\ 0.330 & 0.670 \end{bmatrix}$.
Each token now contains 67% of itself and 33% of the other — soft mixing.

17.5. Positional encoding

Self-attention is permutation-invariant; add positional info either with sinusoidal encodings:

$$PE_{(\text{pos},2i)} = \sin(\text{pos}/10000^{2i/d}),\ PE_{(\text{pos},2i+1)} = \cos(\dots),$$

or with learned position embeddings.

17.6. Transformer block

LayerNorm → Multi-Head Attention → residual → LayerNorm → Feed-Forward (2-layer MLP, expand then project) → residual.

17.7. Three families

- **Encoder-only** (BERT): masked LM, classification, embeddings.
- **Decoder-only** (GPT, LLaMA): causal LM with masked self-attention.
- **Encoder-Decoder** (T5, original Transformer): seq-to-seq with cross-attention.

17.8. Why transformers win

Parallelism (no time-step dependency at training), long-range context, scales with data & compute, unified architecture across modalities (vision ViT, audio Whisper, multimodal CLIP).

18. Topic 16 — Self-Supervised Learning & Foundation Models

18.1. Self-Supervised Learning (SSL) — definition

Learn from *unlabelled* data by creating supervised signals from the data itself. Examples:

- **Masked LM** (BERT): mask 15% of tokens; predict them.
- **Causal LM** (GPT): predict next token.
- **Contrastive learning** (SimCLR, CLIP): pull positives together, push negatives apart.
- **Image patch prediction** (MAE, DINO).

18.2. Contrastive loss (e.g. InfoNCE)

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_i^+)/\tau)}{\sum_j \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}.$$

τ is the temperature. Pull embedding of an image and its augmentation together, push away from other images.

Example 18.0 — InfoNCE numerical

Anchor embedding $\mathbf{z} = [1, 0]$, positive $\mathbf{z}^+ = [0.9, 0.1]$, two negatives $\mathbf{z}_1 = [0, 1], \mathbf{z}_2 = [-1, 0]$.
Cosine similarity ($\mathbf{z}$ already unit, normalise others):
$\mathbf{z}^+/\|\mathbf{z}^+\| \approx [0.994, 0.110] \Rightarrow$ sim $= 0.994$. Negatives are unit already: similarities 0 and -1.
With $\tau = 1$: numerator $= e^{0.994} \approx 2.702$. Denominator $= e^{0.994} + e^0 + e^{-1} = 2.702 + 1 + 0.368 = 4.070$.
$\mathcal{L} = -\log(2.702/4.070) = -\log(0.664) \approx 0.410$.
Lowering τ (e.g. 0.5) sharpens the distribution and grows $\mathcal{L}$ if positives aren't dominant.

18.3. Foundation models

- Pre-trained on broad data, used for many downstream tasks (zero-shot or few-shot).
- LLMs: GPT-4, LLaMA, Mistral, Gemini, Claude.
- Vision: CLIP, DINO, SAM.
- Multimodal: GPT-4V, Gemini, Llama-3-Vision.

18.4. Fine-tuning strategies

Full fine-tuning, head-only, adapters, LoRA, prompt tuning. Trade-off: param efficiency vs adaptability.

19. Topic 17 — Loss Design and Metric Learning

Example 19.1 — center loss numerical

Class 1 centre $\mathbf{c}_1 = [0, 0]$, sample's feature embedding $\mathbf{f} = [0.5, 1.0]$ (true class $y = 1$).
$\mathcal{L}_{\text{geom}} = \frac{1}{2}\|\mathbf{f} - \mathbf{c}_1\|^2 = \frac{1}{2}(0.25 + 1) = 0.625$.
Gradient of $\mathcal{L}_{\text{geom}}$ w.r.t. $\mathbf{f}$ is $(\mathbf{f} - \mathbf{c}_1) = [0.5, 1.0]$ — the gradient pulls the embedding back toward the centre. After one step with $\eta = 0.5$: $\mathbf{f} \leftarrow [0.5, 1.0] - 0.5[0.5, 1.0] = [0.25, 0.5]$.
The centre $\mathbf{c}_1$ is updated separately as a moving average toward feature embeddings of class-1 samples.

19.1. Loss design for class separation (PYQ EndSem Q4)

Goal: classify C classes *and* make embedded clusters compact and well-separated.

- **Base term — cross-entropy:** $\mathcal{L}_{\text{CE}} = -\log \hat{y}_{\text{true}}$.
- **Intra-class compactness — center loss:** $\mathcal{L}_{\text{geom}} = \frac{1}{2}\|\mathbf{f} - \mathbf{c}_y\|^2$ ($\mathbf{c}_y$ = learned class centre; pulls features toward their centre).
- **Inter-class separation — pairwise penalty:** $\mathcal{L}_{\text{inter}} = \sum_{i \neq j} \frac{1}{\|\mathbf{c}_i - \mathbf{c}_j\|^2}$ (penalises close centres).
- **Total:** $\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_1 \mathcal{L}_{\text{geom}} + \lambda_2 \mathcal{L}_{\text{inter}}$.

- All differentiable; $\mathcal{L}_{\text{inter}}$ requires multi-class info (per-batch update of centres).
- **Better mini-batch sampling:** class-uniform — sample K instances from each class so that every centre gets updated and every $\|\mathbf{c}_i - \mathbf{c}_j\|$ pair appears in the batch.

19.2. Metric learning (PYQ EndSem Q3)

We want a transformation $\mathbf{z}_i = L\mathbf{x}_i$ to make same-class points closer.
Distance: $d_L(\mathbf{x}_i, \mathbf{x}_j) = \|L(\mathbf{x}_i - \mathbf{x}_j)\|^2 = (\mathbf{x}_i - \mathbf{x}_j)^\top L^\top L(\mathbf{x}_i - \mathbf{x}_j)$.
For class A (compactness):

$$\min_L \sum_{i,j \in S_A} \|L(\mathbf{x}_i - \mathbf{x}_j)\|^2.$$

Pitfall: $L = \mathbf{0}$ minimises trivially. Constrain $\|L\|_F^2 = 1$ or $L^\top L = I$.

Mahalanobis form: $d_A(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i - \mathbf{x}_j)^\top A(\mathbf{x}_i - \mathbf{x}_j)$ matches d_L if $A = L^\top L$.

Example 19.0 — Mahalanobis distance numerical

$\mathbf{x}_i = [1, 2]^\top, \mathbf{x}_j = [3, 1]^\top$, $A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$ (so feature 1 weighted twice).
$\mathbf{x}_i - \mathbf{x}_j = [-2, 1]^\top$.
$d_A = (\mathbf{x}_i - \mathbf{x}_j)^\top A(\mathbf{x}_i - \mathbf{x}_j) = [-2, 1] \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} [-2, 1]^\top = [-4, 1] \cdot [-2, 1]^\top = 8 + 1 = 9.$
Plain Euclidean^2 is $4 + 1 = 5$. Mahalanobis stretches feature 1 by factor $\sqrt{2}$.
Connection: $A = L^\top L$ with $L = \begin{bmatrix} \sqrt{2} & 0 \\ 0 & 1 \end{bmatrix}$. Then $\mathbf{z}_i = L\mathbf{x}_i = [\sqrt{2}, 2]^\top$, $\mathbf{z}_j = [3\sqrt{2}, 1]^\top$, and $\|\mathbf{z}_i - \mathbf{z}_j\|^2 = (2\sqrt{2})^2 + 1^2 = 8 + 1 = 9$ ✓.

Margin between classes A and B:

$$\max_{L,m} m \text{ s.t. } d_L(\mathbf{x}_i, \mathbf{x}_k) - \max_{p,q \in S_A} d_L(\mathbf{x}_p, \mathbf{x}_q) \geq m, \quad \forall i \in S_A, k \in S_B, \ \|L\|_F^2 = 1.$$

Or trade-off: $\min_L \sum_{i,j \in S_A} d_L - \lambda \sum_{i,k \in S_A, S_B} d_L$ s.t. $\|L\|_F^2 = 1$.

19.3. Weighted Euclidean (PYQ MCQ)

$\tau(\mathbf{x}, \mathbf{y}) = (\mathbf{x} - \mathbf{y})^\top A(\mathbf{x} - \mathbf{y})$.

- (a) A non-identity diagonal: scaled Euclidean. **TRUE.**
- (c) $A = \sum_i \mathbf{u}_i \mathbf{u}_i^\top$ with $k < d$ orthonormals: equivalent to projecting via $\mathbf{x}' = W\mathbf{x}$, W has $\mathbf{u}_i^\top$ as rows. **TRUE.**
- (d) A symmetric: $\tau(\mathbf{x}, \mathbf{y}) = \tau(\mathbf{y}, \mathbf{x})$ (by symmetry of A). **TRUE.**
- (b) Rank-deficient A: τ is still computable (it's a quadratic form), but is no longer a true distance. **FALSE.**
- Answer: **ACD**.

20. Topic 18 — Other Topics That Show Up in Q1

20.1. Document representation

One-hot vectors for words; sum over words gives the histogram (bag-of-words).

- $\mathbf{x} = \sum_i \mathbf{w}_i \in \mathbb{R}^d$ is the *frequency vector* (independent of doc length only *after* normalisation).

- $\sum_i x_i = P$ (total word count).
- Invariant to word order (paraphrase by re-ordering).
- Not invariant to vocabulary order ⇒ Euclidean distance between two histograms *is* invariant to vocab ordering (same permutation applied to both).
- Not invariant to synonyms (replacing word i with word j changes the histogram).

20.2. Distance-preserving linear transforms

$\mathbf{x}' = A\mathbf{x}$.

- A **permutation matrix:** preserves all pairwise distances. TRUE.
- $A = \rho I$**:** a scaling. The decision rule $\text{sign}(\mathbf{w}^\top \mathbf{x})$ changes input by a constant factor; if $\rho > 0$, predictions are unchanged. If $\rho < 0$, all predictions flip but *accuracy* stays the same in a binary problem (just relabel). **TRUE** (accuracy unchanged).

20.3. Auto-encoders and why loss isn't zero

- Bottleneck width < input width: *can't* represent every input perfectly ⇒ reconstruction loss has a floor.
- Even otherwise, training is non-convex; gradient descent finds a local minimum, not the global zero.

21. Worked Past-Paper Solutions (Quiz 1, MidSem, Quiz 2)

21.1. Quiz 1 (Set 1 — Circle): full solutions

1. Vector $[-4, 3, 0, 0]$: $L_0 = 2, L_1 = 7, L_2 = 5, L_\infty = 4$.
2. $\min \mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w}$ s.t. $\|\mathbf{w}\| = 1$: smallest eig. of $\mathbf{X}^\top \mathbf{X}$.
3. Eigenfaces, 1000 images of 100×100: $\mathbf{\Sigma}$ is 10000×10000. Practical rank $\ll \min(N, d)$.
4. Spam $\theta = 0.5$: TP=5, FP=1, FN=1, $P = R = 5/6 \approx 0.83$.
5. 1-NN Manhattan: 3 of 5 correct ⇒ 60%.
6. SVD ($m > n$): (a) Columns of V, (b) Rows-and-Columns, (c) both $AA^\top$ and $A^\top A$ (singular-value sense).
7. Mean-centering: $\boldsymbol{\mu} = [1, 1]$;
   $D' = \{[1, 2], [2, 1], [1, -4], [-1, 2], [-1, -1], [-1, -1], [-2, 0]\}$.
8. MSE: $J(\mathbf{w}) = \frac{1}{N} \sum (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$.
9. PCA grad ascent: $\mathbf{v}^{k+1} = (I + \eta \mathbf{\Sigma})\mathbf{v}^k$.
10. Cov on $y = -x$: $\lambda_2 = 0$, $\mathbf{\Sigma}$ singular.
11. GD on $L = 2w^2 - 4w + 7$, $w^0 = 2, \eta = 0.2$: $w^1 = 1.2, w^2 = 1.04$; optimum $w^* = 1, L^* = 5$.
12. Regression $\{(1, 2), (2, 3), (3, 4)\}$: $\mathbf{w}^* = [1, 1]^\top$, line $y = 1 + x$.

21.2. MidSem — outline of the four big questions

- **Q1.1 Covariance:** $\boldsymbol{\Sigma} = \frac{1}{N}\sum(\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^\top$.
- **Q1.2 Transform W:** $W = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$.
- **Q2.1 Bayes balls:** $P(\mathrm{I} \mid W) = \frac{w_1/(w_1+b_1)}{w_1/(w_1+b_1)+w_2/(w_2+b_2)}$.
- **Q2.3 GD on $f(w) = w^2 + 2w + 1$, $w^0 = 5, \eta = 0.1$:**
  $f'(w) = 2w + 2$; $w^1 = 5 - 0.1 \cdot 12 = 3.8$; $w^2 = 3.8 - 0.1 \cdot 9.6 = 2.84$. Better η: any $\eta \in (0.1, 1)$, e.g. $\eta = 0.3$. One-step $\eta^* = 0.5$ $(1/f'' = 1/2)$; from $w^0 = 5$: $w^1 = 5 - 0.5 \cdot 12 = -1 = w^*$.
- **Q3.1 K-Means** (already worked above).
- **Q3.2 Eigenfaces** (recipe in §5.6).

21.3. Quiz 2 — outline

- **TRUE/FALSE:** BP-global-min FALSE (non-convex), zero-init FALSE (symmetry), CE-for-regression FALSE (CE is for classification).
- **Update equations:** plain GD, momentum, Newton (in §13.1).
- **Z-score normalisation:** $D = \{43, 45, 49, 51, 56, 57\}$. $\mu = 50.17$ (or compute exactly: $\mu = 301.5 \div 6$); the rubric uses $D = \{43, 45, 49, 51, 55, 57\} \Rightarrow \mu = 50, \sigma_{\text{pop}} = 5$, $D_{\text{norm}} = \{-1.4, -1, -0.2, 0.2, 1, 1.4\}$ *Always check the exact dataset variant given to you.*
- **Breadth questions:** Yann LeCun (or similar) for the "left Meta" Q; ReLU = Rectified Linear Unit; NVIDIA for GPUs.
- **MLP params (no bias) 2-8-1:** $2 \cdot 8 + 8 \cdot 1 = 24$.
- **MLP forward + backprop:** as in §12.3.
- **Perceptron (Variant 3):** as in §11.2.

22. Worked Spring-2025 EndSem PYQ (full)

22.1. Q1 — Twelve short answers

1. Cross entropy for $\hat{y} = [0.2, 0.1, 0.7, 0.0]$, target $[0, 0, 1, 0]$: $L = -\log(0.7) \approx 0.357$.
2. 1D conv: $C_{\text{in}} = 3, k = 7, C_{\text{out}} = 5, s = 3$, no bias.
   Params $= 7 \cdot 3 \cdot 5 = \boxed{105}$. (With bias: 110.)
3. "Adding momentum *removes* local minima" is false. Correct: "Adding momentum *reduces* (helps escape) local minima."
4. Why not $\partial L/\partial w_{ij} = 0$ analytically? Solving a non-linear system in millions of variables is infeasible. Backprop is the practical iterative alternative.
5. Why not init all weights equal? Symmetry between hidden units is never broken; all neurons learn the same feature; effective capacity collapses to one neuron per layer.
6. Auto-encoder loss not zero? Bottleneck cannot represent the full input distribution losslessly; plus non-convex optimisation only finds local minima.
7. `Conv1d(1,5,7,padding=3,bias=True)` on $(12, 1, 100)$: params $= 7 \cdot 1 \cdot 5 + 5 = 40$; output $(12, 5, 100)$.

8. Two FC layers, sizes (10,4) no bias and (4,2) with bias: $40 + 10 = 50$.
9. Spam classifier with same scores as Quiz 1, but here for $\theta = 0.5$ get the values you computed earlier ($P = R = 5/6$); for $\theta = 0.2$ get $P = 6/8 = 0.75, R = 6/6 = 1$.
10. Weighted Euclidean — answer **ACD**.
11. Bayes balls — answer **D** (matches the formulaic answer above).
12. Self-supervised learning: "learn from unlabelled data by inventing supervised signals from the data itself, e.g. predict masked words (BERT), next token (GPT), or contrast augmented views (SimCLR/CLIP)."

22.2. Q2.1 — MLP (4 inputs, 5 hidden, 1 output)

W_1: odd rows $+1$, even rows -1. W_2: all $+1$. ReLU hidden, linear output. Bias everywhere.

(a) **Trainable weights:** W_1 is $5 \times 4 = 20$, W_2 is $1 \times 5 = 5$, hidden bias 5, output bias 1. **Total = 31**.

(b) Input $\mathbf{x} = [1, 2, 1, 4]$. Each odd row of W_1 gives $1 + 2 + 1 + 4 = 8$; each even row gives $-(1+2+1+4) = -8$. Pre-activations $[8, -8, 8, -8, 8]$ (rows 1,3,5 odd; 2,4 even). Plus bias (assume 0 unless specified). ReLU $\rightarrow [8, 0, 8, 0, 8]$. Output $= 8 + 0 + 8 + 0 + 8 = 24$, plus output bias $0 \Rightarrow o = 24$. *Caveat: if biases are non-zero you'd add them; the prof's problem says "all neurons have bias" but doesn't give values — assume 0 or write b symbolically.*

(c) Loss $= (15 - 24)^2 = 81$.

(d) Let v = weight from x_1 to first hidden neuron ($w_{11}^{(1)}$). $\partial L/\partial v = 2(o - y) \cdot 1 \cdot u_1 \cdot \mathbf{1}[p_1 > 0] \cdot x_1 = 2 \cdot 9 \cdot 1 \cdot 1 \cdot 1 = 18$.

(e) $\eta = 0.1$: $v_{\text{new}} = v - 0.1 \cdot 18 = 1 - 1.8 = -0.8$.

22.3. Q2.2 — Multi-head attention (worked in §16.4)

22.4. Q2.3 — SGD pseudocode (in §13.6)

22.5. Q2.4 — Loss design for 6-class separation (in §17.1)

22.6. Q3.1 — SVM derivation (in §8.1)

22.7. Q3.2 — Kernel proofs and finding ϕ for $(\mathbf{p}^\top \mathbf{q})^3$

Already in §8.4 and §8.5.

22.8. Q3.3 — Metric learning (in §17.2)

22.9. Q3.4 — Gradient descent on $f(w) = w^2 + 2w + 1$

$f' = 2w + 2, f'' = 2$. From $w^0 = 5$: $w^1 = 5 - 0.1 \cdot 12 = 3.8$; $w^2 = 3.8 - 0.1 \cdot 9.6 = 2.84$. Better η: $\eta = 0.5 \Rightarrow w^1 = 5 - 0.5 \cdot 12 = -1 = w^*$ (one step). **General one-step rule:** $\eta^* = 1/f''(w)$, the Newton step for a quadratic.

23. Practice Problems (with answers)

23.1. Norms / linear algebra

1. Find all four norms of $\mathbf{x} = [3, -4, 12, 0]$.
   Ans: $L_0 = 3, L_1 = 19, L_2 = 13, L_\infty = 12$.

2. Compute $\det\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}$ and find both eigenvalues.
   Ans: $\det = -2$. $\lambda^2 - 5\lambda - 2 = 0 \Rightarrow \lambda = (5 \pm \sqrt{33})/2 \approx 5.37, -0.37$.
3. For the matrix $A = 2I_3$, write its SVD.
   Ans: $U = I, D = 2I, V = I$.

23.2. Regression / GD

1. Fit a line through $\{(0,1),(1,3),(2,5)\}$.
   Ans: $y = 1 + 2x$.
2. Minimise $f(w) = 3w^2 + 6w + 2$ from $w^0 = 4$ with $\eta = 0.1$.
   Ans: $f' = 6w + 6$. $w^1 = 4 - 0.1 \cdot 30 = 1$; $w^2 = 1 - 0.1 \cdot 12 = -0.2$. One-step $\eta^* = 1/6 \approx 0.167$.
3. For ridge regression on the same dataset with $\lambda = 1$, what changes in the closed form?
   Ans: Replace $(\mathbf{X}^\top\mathbf{X})^{-1}$ with $(\mathbf{X}^\top\mathbf{X} + \lambda I)^{-1}$. Numerically, the regression line shrinks toward the origin.

23.3. PCA / SVD

1. Data $\{(1,1),(2,2),(3,3),(4,4),(5,5)\}$. What is the covariance and what are its eigenvalues?
   Ans: $\boldsymbol{\mu} = (3,3)$, $\boldsymbol{\Sigma} = \frac{1}{5}\sum(\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^\top = 2 \cdot \begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}$. Eigenvalues $\{4, 0\}$. Rank deficient because the data is collinear.
2. For 1000 images of 200×200, what's the size of $\boldsymbol{\Sigma}$? And the practical rank if there are 50 individuals?
   Ans: $\boldsymbol{\Sigma}$ is 40000×40000. Practical rank is close to 50.

23.4. Bayesian / probability

1. Bag I has 3 W, 7 B; Bag II has 9 W, 1 B. Drawn ball is white. $P(\text{Bag I} \mid W)$?
   Ans: $\frac{0.3}{0.3+0.9} = \frac{1}{4} = 0.25$.
2. For $\mathcal{N}(0,1)$ vs $\mathcal{N}(4,4)$ with equal priors, is the threshold closer to 0 or 4?
   Ans: Closer to 0 (the tighter class).

23.5. SVM / kernels

1. Find the support vectors of $\{(0,+),(1,+),(2,-),(3,-)\}$ and the decision rule.
   Ans: SV $= \{1, 2\}$. Decision boundary at 1.5. $f(x) = \text{sign}(1.5 - x)$.
2. Show that $\kappa(\mathbf{p}, \mathbf{q}) = (\mathbf{p}^\top\mathbf{q})^2$ corresponds to $\phi(\mathbf{p}) = [p_1^2, \sqrt{2}p_1p_2, p_2^2]^\top$ for 2D inputs.

23.6. NN

1. How many params in MLP $5 \to 10 \to 4$ with bias? *Ans:* $5 \cdot 10 + 10 + 10 \cdot 4 + 4 = 104$.
2. A 2D-input MLP with one hidden layer of 3 ReLU units, output linear. Forward: $\mathbf{x} = [1, -1]$. $W_1 = \begin{bmatrix}1 & -1\\ 2 & 0\\ -1 & 1\end{bmatrix}, \mathbf{b}_1 = 0$; $W_2 = [1,1,1], b_2 = 0$.
   Pre-act: $[1+1, 2+0, -1-1] = [2, 2, -2]$. ReLU: $[2, 2, 0]$. Output: 4.
3. Conv2D layer, $C_{\text{in}} = 3, C_{\text{out}} = 64, k = 3$, padding 1, stride 1. Input $32 \times 32 \times 3$. Params? Output shape?
   Ans: Params $= 3 \cdot 3 \cdot 3 \cdot 64 + 64 = 1792$. Output $32 \times 32 \times 64$.

23.7. Transformers

1. For a 1-head attention on inputs of size $d = 64$ with sequence length $n = 32$, dimensions of Q, K, V?
   Ans: all 32×64 (or $32 \times d_k$ with chosen d_k).
2. Why divide $QK^\top$ by $\sqrt{d_k}$?
   Ans: keeps logit variance ≈ 1 as d_k grows; prevents softmax saturation.

24. Final Cheat Sheet (condensed)

Norms: L_0, L_1, L_2, L_∞.
Closed-form: $\mathbf{w} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$.
Ridge: $(\mathbf{X}^\top\mathbf{X} + \lambda I)^{-1}\mathbf{X}^\top\mathbf{y}$.
LASSO: adds $\lambda\|\mathbf{w}\|_1$, sparsifies.
PCA: eigvecs of $\mathbf{\Sigma} = \frac{1}{N}\mathbf{X}_c^\top\mathbf{X}_c$.
SVD: $A = UDV^\top$.
$\min \mathbf{x}^\top M\mathbf{x}\ \|\mathbf{x}\| = 1$**:** smallest eigvec of M.
Bayes: $P(\omega \mid \mathbf{x}) \propto p(\mathbf{x} \mid \omega)P(\omega)$.
Two Gaussians, unequal σ: threshold near tighter class.
Softmax: $e^{z_i}/\sum_j e^{z_j}$.
CE one-hot: $-\log \hat{y}_{\text{true}}$.
SVM primal: $\min \frac{1}{2}\|\mathbf{w}\|^2$ s.t. $y_i(\mathbf{w}^\top\mathbf{x}_i + b) \geq 1$.
Soft margin: $+C\sum\xi_i$, $y_i(\dots) \geq 1 - \xi_i$.
Kernel PSD: $\mathbf{c}^\top K\mathbf{c} = \|\sum c_i\phi(\mathbf{x}_i)\|^2 \geq 0$.
K-Means: $J = \sum_k \sum_{x \in C_k} \|x - \mu_k\|^2$, non-convex.
Perceptron: update on misclassified only, $\mathbf{w} + \eta y\mathbf{x}$.
MLP backprop: chain $\partial L/\partial o \to \partial o/\partial s \to \partial s/\partial q \to \partial q/\partial p \to \partial p/\partial w$.
ReLU': $\mathbf{1}[p > 0]$.
Linear MLP=SLP: $o = (W^\top\mathbf{u})^\top\mathbf{x}$.
Newton on quad: $\eta^* = 1/f''(w) = 1/(2a)$.
GD with momentum: $\mathbf{v} \leftarrow \beta\mathbf{v} + \nabla L$, $\mathbf{w} \leftarrow \mathbf{w} - \eta\mathbf{v}$.
BatchNorm: $\hat{x} = (x - \mu_B)/\sqrt{\sigma_B^2 + \varepsilon}$, learn γ, β.
Dropout: regulariser, not compression.
Conv params: $k_h k_w C_{\text{in}} C_{\text{out}}$ ($+C_{\text{out}}$ bias).
Conv output: $\lfloor (L + 2P - k)/s \rfloor + 1$.
Self-attn: $\text{softmax}(QK^\top/\sqrt{d_k})V$.
Multi-head: concat h heads $+ W_O$.
LSTM: forget/input/output gates + cell state C_t.
SSL: create labels from data (mask, contrast, predict next).
Foundation models: pre-train large, fine-tune small.
Confusion: P=TP/(TP+FP), R=TP/(TP+FN).
Z-score: $(x - \mu)/\sigma$.
Cross-validation: k-fold averaged, never touch test set early.
Symmetry-breaking: *don't* init all weights equal.
XOR: need hidden layer (not linearly separable).
Residuals: $h = f(x) + x$ saves vanishing gradient.
LeakyReLU: αz for $z < 0$, fixes dead-ReLU.
Adam: momentum + RMSProp + bias correction.
NAND from SLP: $w_0 = 1, w_1 = -1, w_2 = -1$ in ± 1 logic.
NN derivative formulas:
sigmoid: $\sigma(1 - \sigma)$
tanh: $1 - \tanh^2$
ReLU: $\mathbf{1}[z > 0]$
softmax+CE: $\hat{y} - y$.

Final advice from the prof's pattern:

- Read *every word*; the same problem with $w^0 = 2$ vs $w^0 = 0$ has different answers.
- Always show the chain rule *symbolically* before plugging in numbers — partial credit.
- Mention dimensions when you write matrices: "$\mathbf{X}$ is $N \times d$, $\mathbf{y}$ is $N \times 1$".
- Define every new symbol you introduce: "where $\eta > 0$ is the learning rate".
- For "one-step" learning rate questions, the answer is always $1/f''(w)$ (Newton step).
- For Q1, write *precise* answers — no working, no derivations. Don't waste time.
- For Q2/Q3, choose the questions you know best *first*; lock in those marks.
- If you blank, write the formula sheet entry and the recipe — you'll often score $\geq 50\%$ for that alone.

All the best — you have everything you need. Do the worked examples by hand, twice. That's the difference.